

University of Economics, Prague
Faculty of Informatics and Statistics



Serverless Computing: Benefits and Challenges

Bachelor Thesis

Study programme: Applied Informatics

Field of study: Applied Informatics

Author: Zuzana Drozdová

Supervisor: doc. Jiří Feuerlicht, Ph.D.

Prague, May 2020

Declaration

I hereby declare that I am the sole author of the thesis entitled “*Serverless Computing: Benefits and Challenges*”. I duly marked out all quotations. The used literature and sources are stated in the attached list of references.

Prague, 2020

.....

Signature

Acknowledgement

I would like to thank to my parents for supporting me during my studies and friends from the university, who were always willing to explain me anything if I did not understand. Also, I hereby wish to express my appreciation and gratitude to the company Aimtec for giving me such a great opportunity to gain experience. I would also like to thank to my supervisor doc. Jiří Feuerlicht, Ph.D. for his suggestions and a great communication.

Abstrakt

Serverless computing je relativně nová a stále se vyvíjející cloudová kategorie, která přináší nový přístup k vývoji a nasazení aplikací. Základní model Serverless computingu Funkce jako Služba (FaaS), dokáže spouštět malé části kódu, aniž by bylo potřeba zabývat se serverem. Cílem této bakalářské práce je představit architektonické designové vzory, které tomuto přístupu vyhovují. Serverless computing má řadu výhod, ale jelikož se jedná o novou technologii, můžeme tak nalézt i určité nevýhody. Tato bakalářská práce se zabývá jak výhodami, tak nevýhodami a ukáže je na konkrétním případě, nebo je vysvětlí na určité situaci. Praktická část se zabývá vývojem prototypové aplikace, která aplikuje dříve popsáné architektonické vzory. Tato práce také popisuje detailní implementaci a zhodnocení této aplikace. Výstupem této bakalářské práce jsou výhody a nevýhody Serverless computingu, architektonické designové vzory a prototypová aplikace.

Klíčová slova

Serverless computing, Funkce jako Služba, AWS, Cloud computing

Abstract

Serverless computing is a relatively new, and still evolving Cloud category, which brings a new approach to the development and deployment of applications. The Serverless fundamental model Function as a Service (FaaS) allows to execute small pieces of code, without dealing with a server. The purpose of this thesis is to explore this topic more deeply, and introduce architectural design patterns, which are suitable for the Serverless approach. This cloud category has benefits, but as it is a new technology, it has also certain drawbacks. This thesis demonstrates these benefits and drawbacks on use cases, or a situation. The purpose of the practical part is to develop a prototype application, which applies the architectural patterns described in the thesis. In the thesis is also described a detailed implementation and then evaluation of the application. The result of this thesis are benefits and drawbacks of Serverless computing, Serverless Architectural Design Patterns and a prototype application.

Keywords

Serverless computing, Function as a Service, AWS, Cloud computing

Contents

1	Introduction	9
1.1	Goals of the Thesis	9
1.2	Methods	9
1.3	Structure of the Thesis	10
1.4	Contribution	10
1.5	Related Work	10
2	Serverless Computing	12
2.1	Definition	12
2.2	Cloud Computing Models	14
2.2.1	Function as a Service (FaaS)	15
2.2.2	Backend as a Service (BaaS)	16
2.3	Properties	17
3	Serverless Architectural Design Patterns and Use Cases	19
3.1	Web Applications	20
3.2	Event-driven Data Processing	24
3.3	Data Lake	24
3.4	Stream Processing	26
3.5	Serverless Workflow	27
3.6	Conversation bots	29
4	Benefits and Drawbacks of Serverless Computing	30
4.1	Benefits	31
4.1.1	Reduced Labor Cost	31
4.1.2	Reduced Risk	32
4.1.3	Reduced Resource Cost	32
4.1.4	Increased Flexibility of Scaling	34
4.1.5	Shorter Lead Time	34
4.2	Drawbacks	35
4.2.1	Latency	35
4.2.2	Cold Starts	36
4.2.3	State	37
4.2.4	Local Testing	37
4.2.5	Loss of Control	38
4.2.6	Debugging	40
5	Prototype Application	41
5.1	Description of the Functionality	41
5.2	Description of the Technology Environment	42

5.3	Implementation	42
5.3.1	Invoking Step Functions	43
5.3.2	Building a Workflow	43
5.3.3	Operating S3 Buckets	46
5.3.4	Transforming XML Files on AWS Lambda Platform	47
5.3.5	Activity Workers	47
5.3.6	Error Handling	49
5.4	Monitoring and Evaluation	50
5.4.1	Monitoring	50
5.4.2	Overall evaluation	51
6	Conclusion	55
	References	56
A	Source Code of the Prototype Application	60
A.1	Step Functions Workflow	60
A.2	Lambda Functions	61
A.2.1	Invoke Step Functions	61
A.2.2	Get File Type	62
A.2.3	Upload XSLT to S3	62
A.2.4	Upload XML to S3	63
A.3	Upload EDI to S3	63
A.3.1	Delete File	64
A.3.2	XSLT Transformation	64
A.4	Activity Worker	71
A.4.1	EDI Transformation	71

List of Figures

2.1	Popularity of the term “Serverless” as reported by Google Trends	12
2.2	Cloud models, Source: [30]	15
3.1	Difference between monolithic and microservice architecture, Source: [35]	19
3.2	Static content hosted on AWS, Source: [36]	20
3.3	Web application basic architecture, Source: [36]	21
3.4	Organization of Lambda functions, Source: Author, Inspired by: [2]	21
3.5	Web application architecture implementing AWS Cognito, Source: [36]	22
3.6	Three-tier design in monolithic architecture, Source: [4]	23
3.7	Three-tier design in microservice architecture, Source: [4]	23
3.8	Event-driven data processing, Source: [36]	24
3.9	Event-driven data processing with Step Functions, Source: [17]	25
3.10	Visualization of the Data Lake pattern, Source: Author, Inspired by: [10]	26
3.11	Processing website metrics, Source: [36]	27
3.12	IoT Stream processing use-case, Source: [18]	28
3.13	Exemplar Serverless workflow, Source: [38]	28
3.14	Chatbot pattern, Source: [36]	29
4.1	Cost comparison per million of requests, Source: [14]	33
4.2	Database access latency in Serverless and Virtual Machine (VM) based architecture, Source: [15]	36
4.3	Cold starts, Source: Author	37
4.4	Duration of Lambda executions, Source: Author	39
5.1	Workflow of the prototype application, Source: Author	45
5.2	Step Functions monitoring, Source: Author	50
5.3	Execution details with visual workflow, Source: Author	51
5.4	Execution event history, Source: Author	52

List of used shortcuts

AWS Amazon Web Services	FaaS Function as a Service
URL Uniform Resource Locator	BaaS Backend as a Service
XML eXtended Markup Language	VM Virtual Machine
EDI Electronic Data Interchange	MQTT Message Queuing Telemetry Transport
JRE Java Runtime Environment	CNFC Cloud Native Computing Foundation
SDK Software Development Kit	NIST National Institute of Standards and Technology
XSLT Extensible Stylesheet Language Transformations	ISO International Organization for Standardization
API Application Programming Interface	CDN Content Delivery Network
REST Representational State Transfer	TLS Transport Layer Security
HTTP HyperText Transfer Protocol	SQL Structured Query Language
HTTPS Hypertext Transfer Protocol Secure	UX User Experience Design
IDE Integrated Development Environment	IoT Internet of Things
EC2 Amazon Elastic Computer Cloud	ETL Extract, Transform, Load
IAM Identity and Access Management	RAM Random Access Memory
JVM Java Virtual Machine	CPU Central Processing Unit
JAR Java Archive	JSON JavaScript Object Notation
SaaS Software as a Service	ASL Amazon States Language
PaaS Platform as a Service	
IaaS Infrastructure as a Service	

1. Introduction

Cloud computing has been rapidly evolving and was adopted by various corporations, companies and individuals. The basic delivery models, i.e. SaaS, PaaS, and IaaS are being extended by a relatively new model, Serverless. Serverless computing brings a new approach to the development and deployment of applications and its fundamental model Function as a Service or FaaS allows us to deploy and run code without operating the server-side of the applications. Architecture of Serverless applications are mostly composed of various functions, where each of the functions executes only a small piece of the code, which happens only when the function is invoked. This approach can be used in various use cases, and promises cost efficiency, because the costs are based on the usage. On the other hand, this approach can also have certain drawbacks, which could be a challenging aspect for the providers. I confronted this topic at work, where I was working on developing an AWS Lambda function and an API Gateway. Later I took a couple of courses, which were focused on the development of Serverless applications using the AWS services. The AWS is a cloud provider and its AWS Lambda function is often mentioned as a starting point of Serverless computing. Therefore, I focus exclusively on the AWS services, even though I am aware of other providers as well. I found this topic very interesting, because I am convinced, that this can be the future of computing.

1.1 Goals of the Thesis

The purpose of this thesis is to introduce the architectural design patterns, which are suitable for the Serverless approach, and discuss the components and services, which are used for the particular pattern. The following goal is to determine benefits and drawbacks of Serverless computing, which are showed on a use case or explained on a situation. The goal of the practical part is to develop a Serverless application, which applies an architectural design pattern described in the theoretical part.

1.2 Methods

The most used method for the theoretical part was a research of studies and books. A book written by Roberts and Chapin [1] discusses various aspects of Serverless computing and it is often referenced in this thesis and also in most of the studies, which are used for this thesis. Other books were focused on the architectural design patterns and implementation[2],[3]. The studies or case studies were essential in supporting the ideas described in this thesis. To explore the various Serverless architectural design patterns were used apart of studies and books also conferences and summits organized by the cloud provider AWS. These conferences are only available in a video form on YouTube.

The practical part requires different methods and materials. The most important aspect was to learn how to work with the AWS services. For this purpose on-line courses, tutorials and documentations were used. The programming languages of the developed functions are Java 8, and JavaScript.

1.3 Structure of the Thesis

The thesis is divided into 6 different chapters, with the first and the last one being the introduction and conclusion. The second chapter focuses on the fundamentals of Serverless Computing, which is a brief history of the term Serverless, definition, putting Serverless into the Cloud context, and the fundamental properties of Serverless Computing. The third chapter introduces Architectural Design Patterns and follows up the key services of each pattern. The fourth chapter discusses the benefits and drawbacks of Serverless computing and some of the ideas are supported by a study or an experiment made by the author. The fifth chapter describes the prototype application, its implementation and the evaluation.

1.4 Contribution

The contribution of this thesis is a summarization of the various patterns that are suitable for Serverless approach. The benefits and drawbacks are described, because we need to realize that Serverless computing can be cost effective, but on the other hand it has also its imperfections, which should be considered and tested.

The contribution of the practical part is to show how Serverless can deal with a state and that it is possible to build an application, its components can be both Serverless and non-Serverless.

The contribution for the the author is to extend the knowledge of Serverless computing and to learn how to build a Serverless application.

1.5 Related Work

As mentioned before, the key book, which is very often referenced in most of the studies as well as in this thesis is written by M. Roberts and J. Chapin [1]. This book discusses the basic aspects of Serverless, which includes the definition of Serverless, the benefits and limitations of Serverless and also determines the Serverless technologies. M. Roberts also published an article [4], where he extends the ideas described in the book. Another article, which explains the essence of Serverless Computing, its benefits and drawbacks, and other aspects is written by J. Fruhlinger [5]. The articles, which briefly describe cloud computing

are: [6], which is the NIST definition of cloud computing, [7], which discusses the characteristics of cloud computing, and [8], which describes the service and deployment models of cloud computing. A whitepaper `citebaaswhite`, which is only available on github, defines Serverless computing, determines use cases and compares Serverless to other cloud models. The architectural design patterns are shown in various works. The cloud provider AWS organizes every year a conference `re:Invent` or summits, where they talk about various aspects of AWS Services. They are available only on youtube. A presentation at the `re:Invent` [9] talks about the Serverless architectural patterns and best practices, and a presentation at the AWS Summit [10] is about architectural pattern Data Lake. Other works related to Serverless architectures are books. One of them is about Serverless design patterns [2], which discusses web application pattern and stream processing pattern, and the other one [3], which describes especially stream processing. J. Daly describes in his article [11] 19 Serverless Microservice patterns. Another sources, which were used for Serverless architectures and especially to clarify the difference between monolithic and microservice architecture, are [12] and [13]. The studies, which were used for proving the benefits or drawbacks are [14], which compares the costs of monolithic, microservice and FaaS architecture, [15], which is focused on improving the latency in Serverless architectures, and [16], which is focused on building a Serverless chatbot. We also use a lot of sources from AWS website ([17], [18], [19], [20], [21], [22], [23]).

2. Serverless Computing

Serverless computing is a new technology and its popularity has been growing since the past few years. One of the first Serverless services, the so called AWS Lambda was initially released in November 2014 [21], which is not a very long time ago. The first usages of the term Serverless can be found in an article called *Why The Future Of Software And Apps Is Serverless* by Kenn Fromm in 2012 [24], in which Fromm claims that *"Even with the rise of cloud computing, the world still revolves around servers. That won't last, though. Cloud apps are moving into a Serverless world, and that will bring big implications for the creation and distribution of software and applications."* In this article the author does not define Serverless computing, but he describes how it could involve.

When AWS Lambda launched in 2014, as previously mentioned, the term became more popular and the popularity grew even more since 2015, when API Gateway by AWS was released [22]. For example, Ant Stanley in his article called *Servers are dead..* claims the following: *"The inexorable move to Serverless architectures took another step forward yesterday when Amazon Web Services announced API Gateway"* [25].

Later that year, in October 2015, the term *serveless* was used as a name of a talk at AWS re:Invent conference called *"The Serverless Company Using AWS Lambda"* [26]. By the year 2016 Serverless becomes a dominant name for this area and was adopted by conferences, article titles or job descriptions.[4]

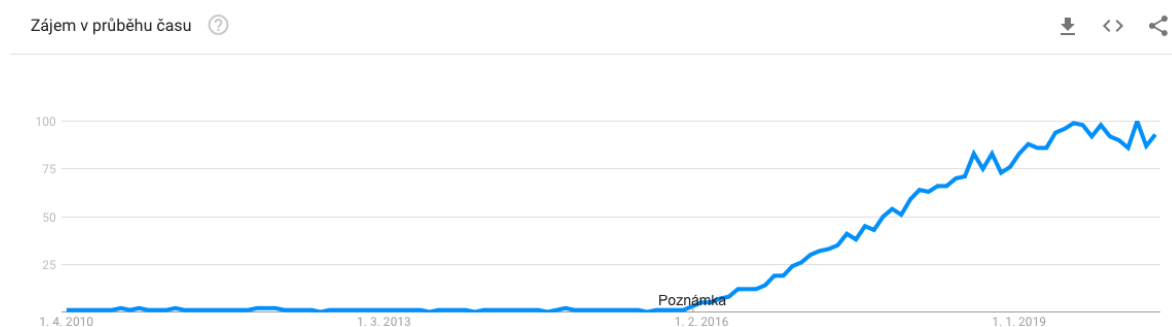


Figure 2.1: Popularity of the term “Serverless” as reported by Google Trends

2.1 Definition

We will take a look at definitions from various organizations and try to understand the concept of Serverless computing. Multiple definitions can be found across the internet, however all of them are quite recent. There are two main world institutions for standardization, the ISO (International Organization for Standardization) and the NIST (National Institute of

Standards and Technology) and surprisingly we can only find definition from one of them, and that is by the ISO from the year 2020.

The definition by ISO is as it follows: *"Cloud service category in which the cloud service customer can use different cloud capabilities types without the cloud service customer having to provision, deploy and manage either hardware or software resources, other than providing cloud service customer application code or providing cloud service customer data"*[27].

ISO claims that Serverless computing is one of the categories of cloud computing. We will shortly discuss cloud computing later to understand the whole concept. This definition has 3 different entries as it follows: *"Serverless computing provides automatic scaling with dynamic elastic allocation of resources by the cloud service provider, automatic distribution across multiple locations, and automatic maintenance and backup., Serverless computing functionality is triggered by one or more cloud service customer defined events and can execute for a limited time period as required to deal with each event. and Serverless computing functionality can be invoked by direct invocation from web and mobile applications* [27].

Another definition is by AWS, which is one of the biggest Serverless service providers and its services are used frequently in this thesis. The definition is: *"Serverless is the native architecture of the cloud that enables you to shift more of your operational responsibilities to AWS, increasing your agility and innovation. Serverless allows you to build and run applications and services without thinking about servers. It eliminates infrastructure management tasks such as server or cluster provisioning, patching, operating system maintenance, and capacity provisioning. You can build them for nearly any type of application or backend service, and everything required to run and scale your application with high availability is handled for you"*[28].

The following definition is by J. Fruhlinger, who writes IT related articles, defines Serverless computing as it follows: *"Serverless computing is an execution model for the cloud in which a cloud provider dynamically allocates—and then charges the user for—only the compute resources and storage needed to execute a particular piece of code"* [5].

These definitions differ in what is Serverless computing. According to the ISO, Serverless computing is a "Cloud service category", AWS talks about Serverless as a native architecture and J. Fruhlinger claims, that Serverless computing is an execution model. All definitions agree on the fact, that Serverless computing provides dynamic scaling, so it uses the necessary compute resources to run all the processes at the same time. AWS and ISO claims that we can use the compute resources by a cloud provider, use different types and build applications without managing servers. Both definitions sound very technical compared to J. Fruhlinger, who is the only one who mentions costs when he says that the user is charged only for the resources, that are needed to execute the code.

2.2 Cloud Computing Models

References about Cloud computing can be found dating back before the 21st century and we could discuss the origin of this term and what it meant in those times. However, we will consider that this term appeared in 2006, when Eric Schmidt said, that there is a new model, which should be called Cloud computing and people do not know yet about its big opportunity [29].

Later, in 2011 the NIST published the following definition: "*Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. This cloud model is composed of five essential characteristics, three service models, and four deployment models*" [6]. We shortly look at the five essential characteristics, three service models and four deployments models that are not described in the definition.

The five essential characteristics are:

- *On-demand self-service* - Additional cloud resources can be provided without a human interaction.
- *Broad network access* - Users can access the model from the internet anywhere and anytime.
- *Multi-tenancy and resource pooling* - Users can share their resources between each other, if they have a permission for that.
- *Rapid elasticity and scalability* - The cloud model is able to scale up or down automatically.
- *Measured service* - Users only pay for what they use.[7]

The three service models are:

- *Software as a Service (SaaS)* - SaaS is an end-user application. As an example of this service model we could give Microsoft Office or Netflix.
- *Platform as a Service (PaaS)* - This is a platform, where developers can build and deploy their own applications.
- *Infrastructure as a Service (IaaS)* - Developers build and deploy their applications, plus they should have a control over the storage and compute resources and they might even control the operating system. [8]

The four deployments models are:

- *Private cloud* - Private cloud is for individuals or businesses and it is not shared with anyone.
- *Public cloud* - Public cloud is for the general public.
- *Community cloud* - Community cloud is for businesses who share the same concerns.
- *Hybrid cloud* - Hybrid cloud can combine two or more models mentioned above. [8]

From what was mentioned above we can say that cloud computing is a model including multiple different models with special characteristics. The models have certain differences, especially we need to focus on SaaS and compare it to the others. SaaS is an end-user application where users can sign up and benefit from the application as a user, so the user does not need to think about how it works and eventually pays for that. Unlike SaaS, the other two models gives the possibility to build and deploy the application and we could consider it more technical, because to make it work correctly we need developers. Depending on the model, we can add additional logic, storage or choose an operating system. At this point we could start using the term Serverless and consider it a new cloud computing service model. This model is often known as Function as a Service (FaaS) or Backend as a Service (BaaS). It is still rather difficult to find sources adding FaaS and BaaS to the Cloud computing models, because it is still rapidly evolving. Some resources even consider FaaS and BaaS being a part of PaaS.

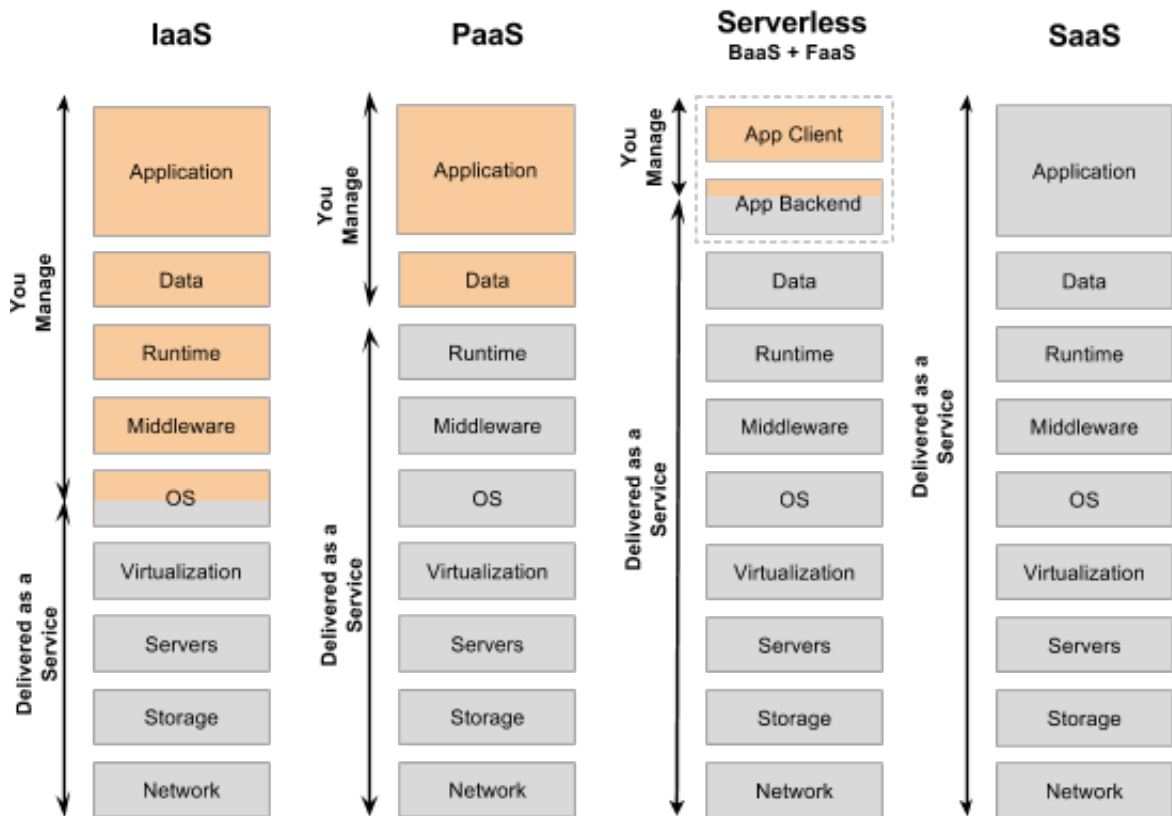


Figure 2.2: Cloud models, Source: [30]

2.2.1 Function as a Service (FaaS)

Function as a Service is a Serverless product able to host and execute code containing a business logic. The execution of the function is event driven, so the function can be triggered by another element, such as when an user clicks on a button in an application [31]. The logic of the function can be written and updated on the fly without having to redeploy the whole

application and the developers can choose between a variety of programming languages. Unlike other cloud models like Software as a Service or a Platform as a Service that are running always even if sitting in idle, FaaS scales on-demand depending on the number of invocations and in case of no invocation, the Serverless workload does not run [32]. FaaS are stateless and should be running a short time. The maximum time limit of the duration of the AWS Lambda execution is set to 15 minutes. FaaS can be defined as AWS describes the AWS Lambda function[23]:

"AWS Lambda lets you run code without provisioning or managing servers. You pay only for the compute time you consume. With Lambda, you can run code for virtually any type of application or backend service - all with zero administration. Just upload your code and Lambda takes care of everything required to run and scale your code with high availability. You can set up your code to automatically trigger from other AWS services or call it directly from any web or mobile app."

As the definition says "You pay only for the compute time you consume.", FaaS are being considered as a cost-efficient way of software development [31].

Using functions as modular components for building a Serverless application is considered as a microservice architecture. Microservice architecture appeals to developers, because they can create and modify those functions and easily implement them into the codebase [31].

2.2.2 Backend as a Service (BaaS)

The term Backend as a Service has been in use for a longer time than Serverless computing. Since 2012 multiple articles can be found about Backend as a Service often with the same meaning as Mobile Backend as a Service (MBaaS). Cloud Native Computing Foundation (CNFC) defines BaaS as it follows: *"Backend-as-a-Service (BaaS), which are third-party API-based services that replace core subsets of functionality in an application. Because those APIs are provided as a service that auto-scales and operates transparently, this appears to the developer to be Serverless"* [33]. According to this definition third-party services manage the server infrastructure and allows to scale the application and provide tools to build a backend for an application. Those tools can be push notifications, storage, database or authentication services. Thanks to this, developers can spend more time on developing the client side or frontend rather than backend.

We can also find opinions, that backend as a service works together with FaaS. Function as a Service gives us the possibility of having multiple functions within one application, but we still need to add something more to have a fully working application. Functions can not store any data, because they contain only code. If we want to work with data we need to have a database, which is one of the examples of BaaS. Another example of BaaS is user authentication and management or push notifications for a user.

2.3 Properties

This section discusses the properties of Serverless computing. The properties consider costs, hosting the Serverless functions or applications, the dynamic scaling depending on load, the state of FaaS models, event-driven functions and the distribution of the Serverless architectures. The properties are described in detail below:

- **Costs:** Costs are precisely based on the usage of Serverless platforms. Having a Lambda function that runs for one minute an hour, we only pay for that one minute. The costs are not only based on the usage, there are also some additional costs for using Serverless platforms in general, but the main cost is proportional to the usage of the services[1].
- **Hostless:** Unlike other ways of operating server-side, Serverless does not require the server management like deploying, running and monitoring an instance of our application. There is no longer a need to manage physical or virtual machines or containers that are normally used for running an application. This does not mean that servers no longer exist, but they are operated and managed by the provider [1]. Hostless applications divert the attention from monitoring host's metrics like memory, CPU, disk-size or others, but on the other side developers need to focus on different monitoring metrics provisioned by the provider for tuning the architecture. Every Serverless platform comes with certain limitations and we need to avoid exceeding them. For instance, AWS Lambda has a limitation of concurrent executions and to make sure we stay under the limit, we can use the monitoring tools. In case of approaching the limit we can adjust the architecture of the application [34].
- **Elasticity:** Serverless application scales automatically and the scaling is dynamic depending on the current requirements. Thanks to elasticity, Serverless application can scale from zero, when there is no load, to the maximum that is allowed by the provider and then again back to zero. The advantage is that developers do not need to manually manage automatic scaling at all unlike other ways of developing [34].
- **Stateless:** Every function is triggered by an event and many BaaS platforms support those events. For instance, a Lambda can be triggered by clicking on an element in a web-application. The benefit of event-driven architecture is a low level of interconnection between particular Serverless components and it is easy to add new components without rebuilding and redeploying the whole application[34].
- **Event-driven:** Every function is triggered by an event and many BaaS platforms support those events. For instance, a Lambda can be triggered by clicking on an element in a web-application. The benefit of event-driven architecture is a low level of interconnection between particular Serverless components and it is easy to add new components without rebuilding and redeploying the whole application[34].
- **Distributed:** We talked about stateless as a property, where we cannot store anything in memory. In most of the applications it is necessary to store data or work with them and for this reason it is essential to implement other services like Backend as a Service. The application may consist of multiple components that also have to be integrated - database, authentication, distributed queue or others. The combination of FaaS and

BaaS leads to a larger granularity of the architecture and the result is a distributed system [34].

3. Serverless Architectural Design Patterns and Use Cases

In this section we will focus on the architectural design patterns of Serverless solutions. We will understand this term as a solution, that can be reused to solve a frequent design Serverless application issue. There are naturally a plenty of solutions how to organize a code or an application and what technologies would possibly be the best to use.

We can observe different approaches when it comes to Serverless architectural patterns as such. The first approach is taking into account the whole application and the second one describes the patterns rather for each technology, or more likely for smaller pieces of a certain application. For instance, at the re:Invent conference in 2017 were introduced three architectural patterns: Web application, Data Lake and Stream Processing. [9] and Jeremy Daly, who is officially "AWS Serverless Hero", describes 19 Serverless patterns [11].

It is important to mention, that Serverless architectures are an approach to microservices [12] as they should meet the criteria of a distributed system and use the resource capacity only when necessary. Unlike monolithic architecture, which is a software application whose modules cannot be executed independently [13], microservice architecture is a distributed software, its components can be executed independently. In the figure 3.1 is a graphical representation of microservice architecture and the monolithic architecture.

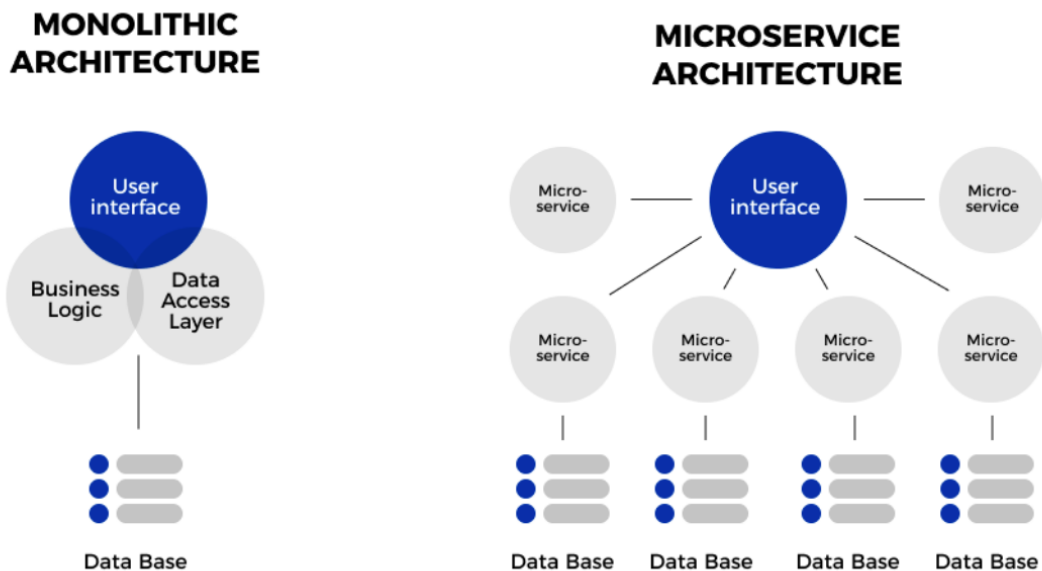


Figure 3.1: Difference between monolithic and microservice architecture, Source: [35]

3.1 Web Applications

Web applications are a common and a popular use case for Serverless computing. Web applications are mostly used by an end-user, so they need an appropriate user interface. The interface is a static content that does not require a compute power. Serverless solution for hosting the static website is S3, which is a storage. S3 may be very slow to deliver the content data to a user, which is a drawback that needs to be fixed. Other issue here is the security. User should not have access directly to an S3 bucket. To solve these issues, it is highly recommended to use AWS CloudFront. CloudFront is a Content Delivery Network (CDN), which improves the delivery of static assets to the user. CloudFront among others, can also manage the cache control and access the static content over a custom domain and provide a Transport Layer Security (TLS) certificate from AWS Certificate Manager[2]. Figure 3.2 represents the pattern for hosting a static website in AWS.

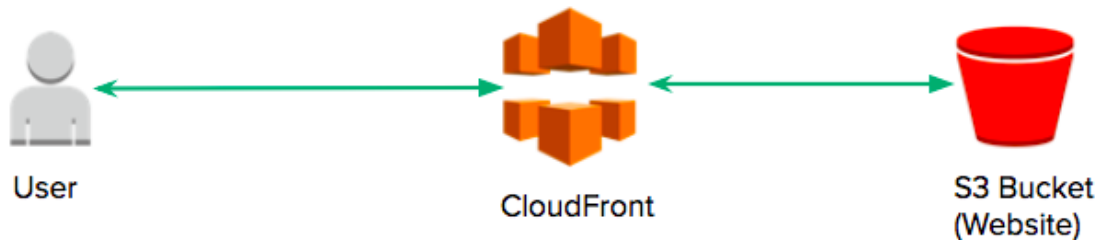


Figure 3.2: Static content hosted on AWS, Source: [36]

Web applications are mostly based on the request/response system, which can be supported by using a rest API. API Gateway by AWS is the right solution as it integrates a REST API (Representational State Transfer Application Programming Interface). API Gateway communicates between the static content and the logic of the application and it is next fundamental component of a web application. API Gateway can scale up to handle any incoming user request[36]. The logic of a web application is composed of any number of Lambda functions and each API method has one Lambda function as an endpoint. When it comes to the functions, a lot of Serverless developers recommend to use as many individual functions as possible with a small amount of code. In any event there are at least two approaches to designing those functions and to mapping the API endpoints. The first approach is to have a Lambda function for each session separately. The other option is to handle more specific tasks within one Lambda function. An example of this could be handling a database. One Lambda function can implement the whole logic such as create, get, update and delete or the same logic can be used for individual functions. The database is also another essential component of the web application[2]. In this case a fully managed No-SQL database DynamoDB will be used. In the figure 3.3 is a representation of the following complements plugged into the architecture. The figure 3.4 demonstrates the two different approaches to design the Lambda functions.

Most of the web applications have a user management and AWS provisions various tools to implement it for our application. Discussing authorization as such, we can authorize to make

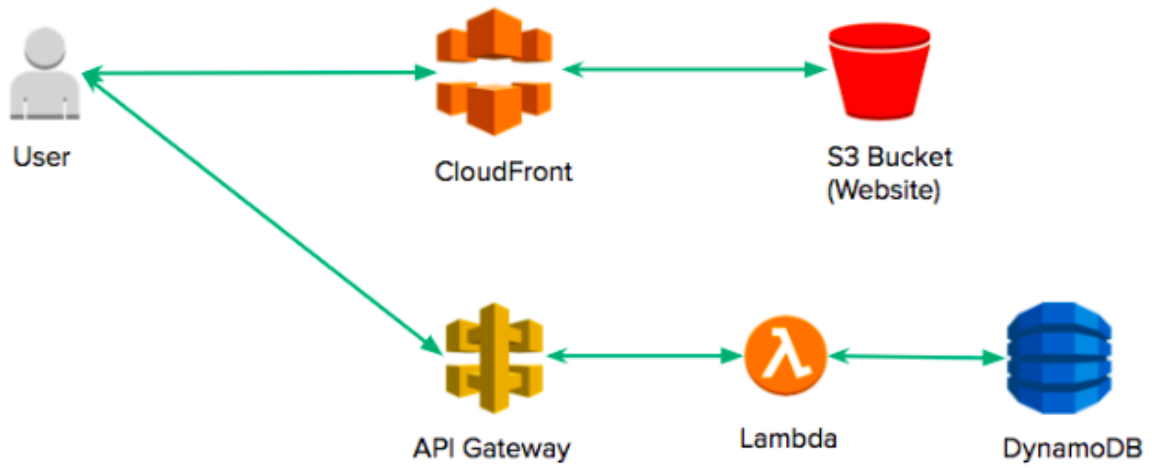


Figure 3.3: Web application basic architecture, Source: [36]

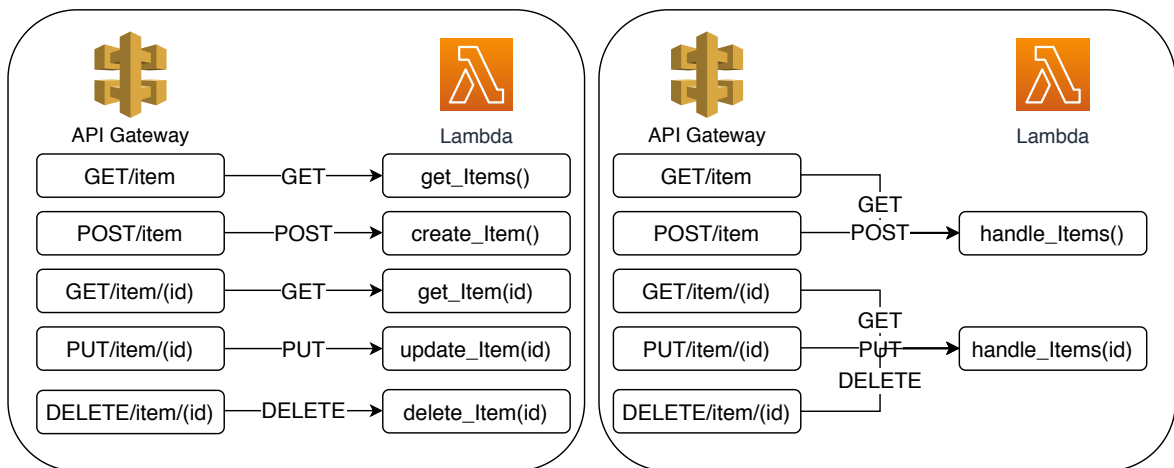


Figure 3.4: Organization of Lambda functions, Source: Author, Inspired by: [2]

a HTTP request using the AWS Signature. For this method we need to provide AWS user's access key and secret key. To obtain these parameters it is necessary to create a user within the AWS account - this is not a convenient method for managing hundreds or thousands of users. Another option is to use a custom authorizer, which can be created directly in the API Gateway console. Custom authorizer offers using either Lambda function or AWS Cognito. Lambda requires a logic that we need to provide. For instance, Basic Auth requires a username and a password and Lambda function has to verify these entries. Unlike custom Lambda function, AWS Cognito provides user management like sign-in, sign-up or access control. A possibility how to manage the users without having them to sign-up, is using an external provider that already contains users and integrate it to AWS Cognito. An example of this can be Google or Facebook. The complete architectural design pattern can be seen in the figure 3.5.

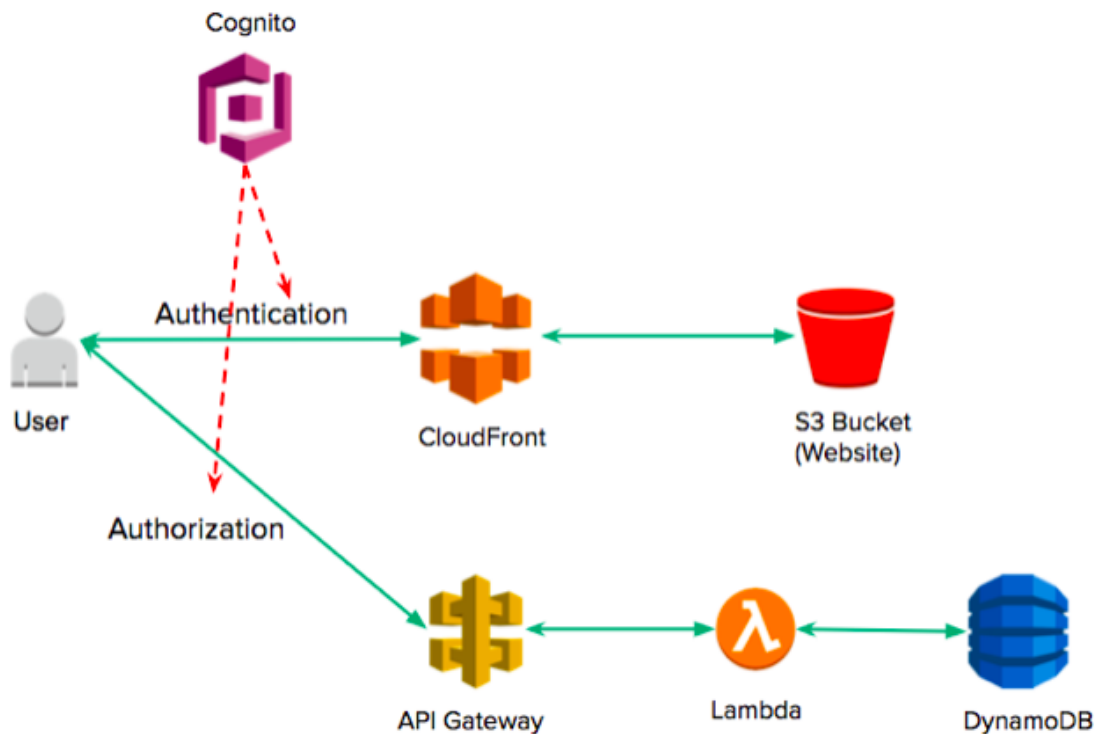


Figure 3.5: Web application architecture implementing AWS Cognito, Source: [36]

This architecture is an approach to a three-tier client-server system, with the server-side logic, which is composed of the presentation layer, logic layer and data layer. Michael Roberts demonstrates the difference between the monolithic architecture and the Serverless microservice architecture [12]. The monolithic approach is represented in the figure 3.6. The representation of the three-tier Serverless design is demonstrated in the figure 3.7.

In contrast to the monolithic architecture, Serverless solution differentiates in [4]:

- The authentication and authorization are not a part of the main logic anymore, instead it is replaced with a BaaS service.
- The client can have a direct access to a fully managed database provisioned by a third party. The security profile for accessing the database will be likely different than when



Figure 3.6: Three-tier design in monolithic architecture, Source: [4]

the server access that database.

- Part of the logic, that was previously in the server, moved to the client side. For instance, the user session tracking or reading from a database.
- Some functionality that we want to keep in the application's logic may be demanding on computing resources and could require access to a considerable amounts of data, an example of this is a search function, which is related to the user experience design (UX). We can move this functionality from the server to a single FaaS function and both the server and the client will be able to read from the same database.
- The "purchase" functionality can be replaced with another FaaS function. We will probably break that functionality into separate functions as demonstrated in the figure 3.7.

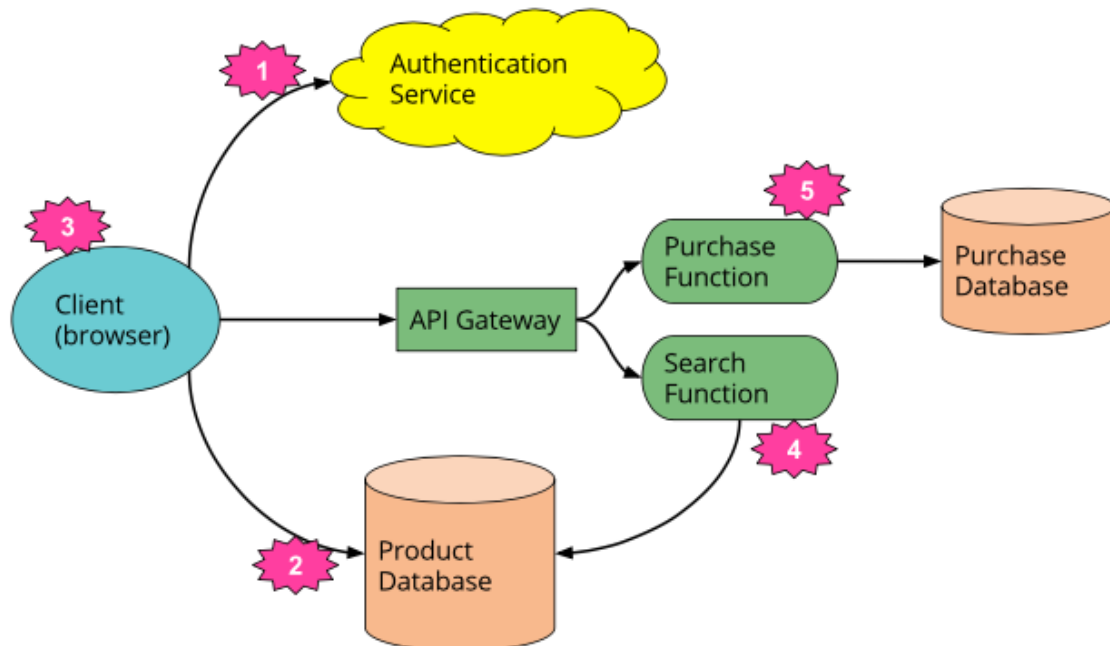


Figure 3.7: Three-tier design in microservice architecture, Source: [4]

3.2 Event-driven Data Processing

A typical use case, that is frequently used to explain this pattern and its design, is image resizing. Unlike web application, this pattern is not that complex and can be used as a part of a Serverless application. Using the image resizing example - user wants to upload a picture to a Serverless application (or even non-Serverless application). The size of the image is too large and therefore is convenient to resize that picture. This functionality can be easily implemented within a single Lambda function.

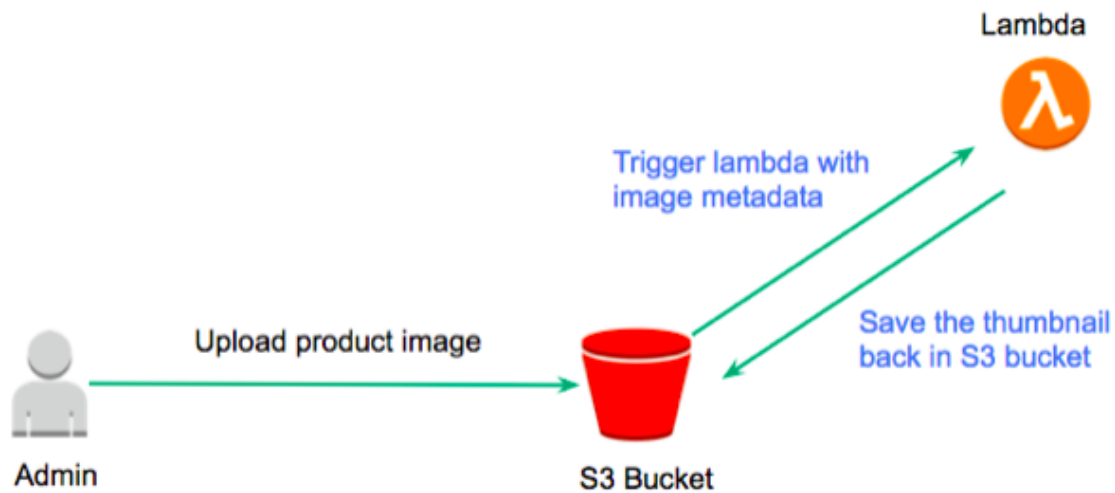


Figure 3.8: Event-driven data processing, Source: [36]

Figure 3.8 represents the architectural design of the Event-driven data processing. The event is when any kind of a file is uploaded to a S3 bucket. This emits S3 PUT event, which triggers the Lambda function. Lambda will apply its functionality to the file and then will put the processed file back to the S3 bucket. Thanks to the scaling trait, Lambda can process a large number of the files at the same time [36].

Another approach to the event-driven data processing is using a state machine, which will be discussed in more detail in the section "Serverless workflow". This can be used in case, that the file processing requires more complex logic and one Lambda function would not be enough. Other reasonable aspect for implementing a state machine is that it is stateful during the whole particular process. The graphical representation of this is in the figure 3.9 according to the AWS Step Function use cases homepage.

3.3 Data Lake

Data Lake is an architectural pattern that allows us to collect and store those data in a centralized location, so they are ready to be categorized, processed, analyzed and consumed by diverse individuals or organizations. Collected data can be data from relational and NoSQL databases or completely unstructured data like log files or Internet of Things (IoT)

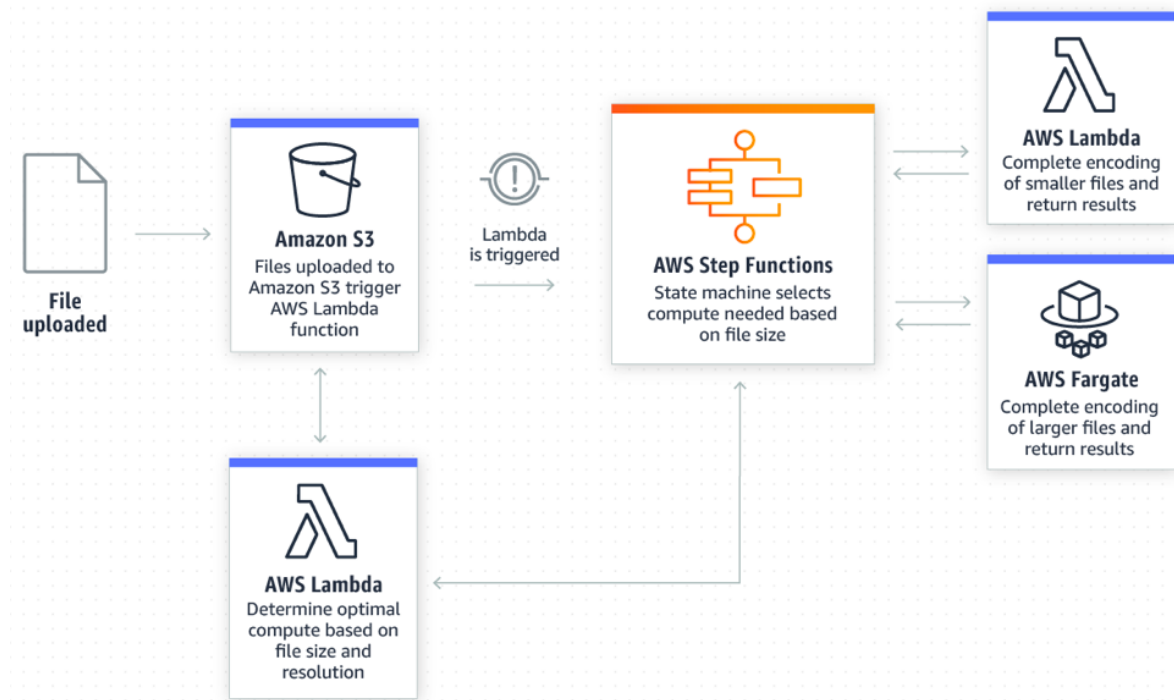


Figure 3.9: Event-driven data processing with Step Functions, Source: [17]

[10]. The architecture of Data Lake might differ depending on the specific use case and can also use different services offered by AWS [9].

In this pattern we need to collect incoming data and store them. For ingesting data serves a tool called Amazon Kinesis Firehose and as a storage is mostly used S3 bucket. The advantage of S3 is that it provides unlimited amount of storage and also provides tools to secure the data. Amazon Kinesis is one of the key services that allows to collect and analyze streaming data and store them in one location. Depending on the use case, Kinesis offers different capabilities, which are Data Streams, Firehose and Data Analytics. Kinesis Data Streams enables to directly collect streaming data, process them with other service, which can be a Lambda and giving us an output. Data Analytics makes it possible to interact with the streaming data using SQL statements. The Amazon Kinesis Firehose is a tool used in this example and it is a service that captures and delivers data into a storage, in this case S3 bucket [3].

Having the data collected and stored, we can go further towards the Catalog and transform. AWS Glue is the most important component for this part of the pattern. AWS Glue is an ETL service, which is a shortcut for extract, transform and load. ETL is a process that firstly extracts data even from multiple sources, transforms them into a proper format and structure and loads the data into another storage. In this example we use a Glue Crawler, that goes into the S3 bucket, scans the data and creates a schema with the data structure and a format. Afterwards it writes the schema into the Glue Catalog. Once the schema is available in the Glue Catalog, the data can be transformed by using a script and stored in another storage containing the processed data. To analyze the data we can use SQL scripts.

Amazon Athena allows us to access the data in S3 without launching a database and analyze them using a custom SQL statements. To visualize the result there is a service called Quick Sight[10].

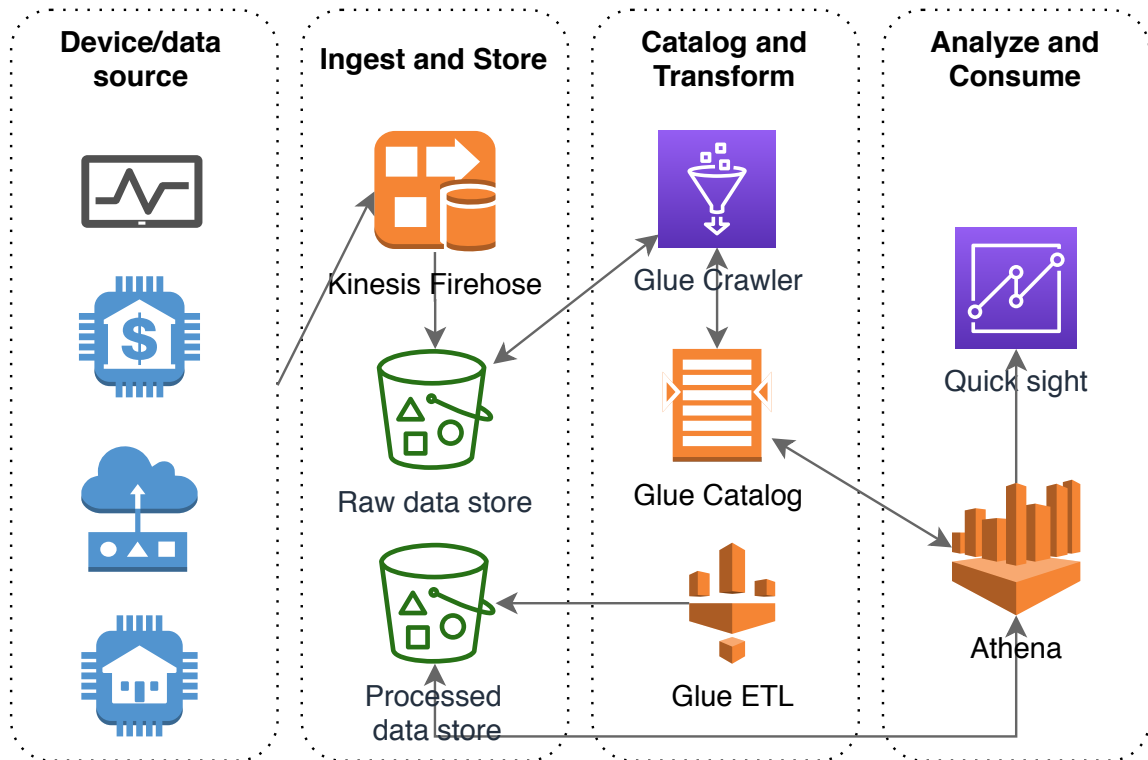


Figure 3.10: Visualization of the Data Lake pattern, Source: Author, Inspired by: [10]

The graphical visualization in the figure 3.10 is an example for this architectural pattern. Depending on various use cases there might be a dozen of different implementations using other tools [9].

3.4 Stream Processing

Stream processing focuses on the data analysis as well as the data lake, but the main difference is that the stream processing is able to directly analyze the data without firstly storing them.[37] This pattern is used especially for real-time analytics. Exemplar use cases are measuring temperature of an IoT device, video face recognition, medical sensors, or websites metrics like the time spent on a certain page. Depending on the use case, the design and the services used for a particular application can differ. First use-case frequently presented is the one with website metrics. Whether it is an e-shop or a streaming service, it is convenient for the providers to get to know and interpret the user's behaviour on their website. This use-case will focus on the web application pattern hosting a website on S3 bucket. AWS Pinpoint is a service allowing to log particular user events on that website and stream them to Kinesis Firehose. Kinesis Data Analytics that can query the data streams using SQL statements can be used for analyzing those events. In this use-case is a Lambda function for consuming the

information and indexing in ElasticSearch [36]. Having the data in ElasticSearch, we can use a visualization tool to visualize the data. In the figure 3.11 we can see a demonstration of this pattern.

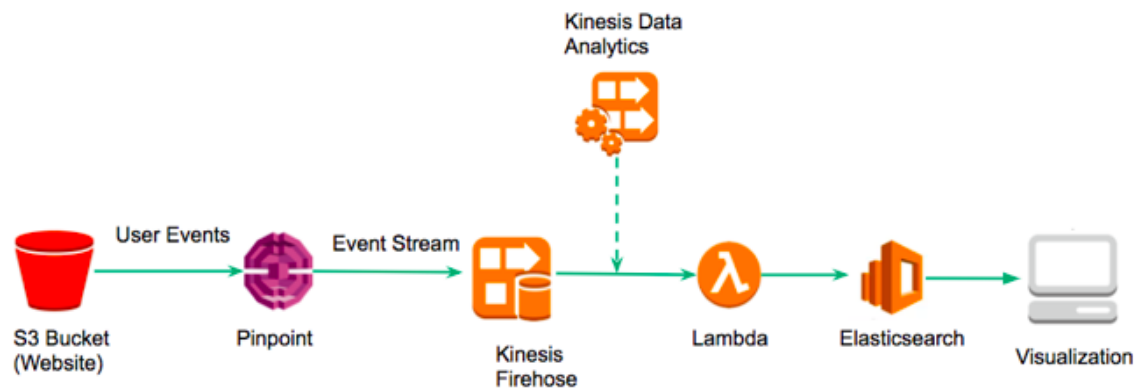


Figure 3.11: Processing website metrics, Source: [36]

A whitepaper publicized by AWS is dedicated to Serverless streaming architectures and elaborates a use-case focused on a medical sensor devices [18]. Those devices are wired to a patient, who is receiving treatment in a hospital. This use-case is more complex when compared to the previous one and the data are streaming constantly from one or more devices over IoT gateway. MQTT protocol forwards those data securely to the AWS IoT service. Within that service we can configure an IoT rule, its purpose is to select a measurement, and IoT action, which delivers these measurements to the Lambda function. The functionality of this Lambda is to anonymize the data, e.g. by removing the patient's name and transmitting the data into the Kinesis Firehose. The next task of the de-identifier is to encrypt the name of the patient and store that information in the DynamoDB along with its ID. Once the data are in Kinesis Firehose, they can be enriched within Lambda function. The Lambda function is triggered by the Kinesis Firehose and looks up information about the devices from DynamoDB and adds them to the records. Kinesis then sends these records to the S3 bucket as a backup and to the Kinesis Data Analytics. This services analyzes the measurements and if it detects any anomaly, it sends the anomaly score to the Kinesis Data Stream. This score is then processed by the Lambda function, which alerts medical staff. The graphical representation can be seen in the figure 3.12.

3.5 Serverless Workflow

Lambda functions are designed to run one single operation without being dependent on the whole logic of the application. Having a complex operation, which includes multiple steps and requires to keep the state within the whole process could be problematic to execute within a single Lambda function. And what also needs to be taken in account, is that Lambda has certain limitations such as 300s timeout. AWS Step Functions allow us to build a workflow, where an output of one step is an input into another one and each step has its own task,

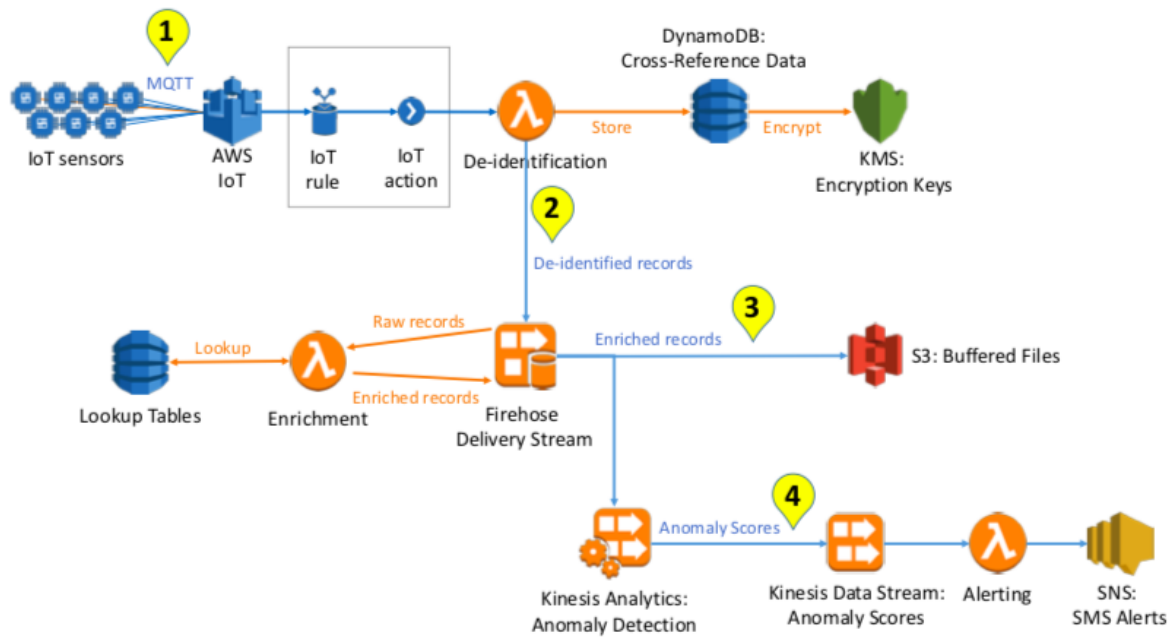


Figure 3.12: IoT Stream processing use-case, Source: [18]

which can be a Lambda function. This workflow can transmit an important information over the whole operation and it can also retry a particular step if it fails.

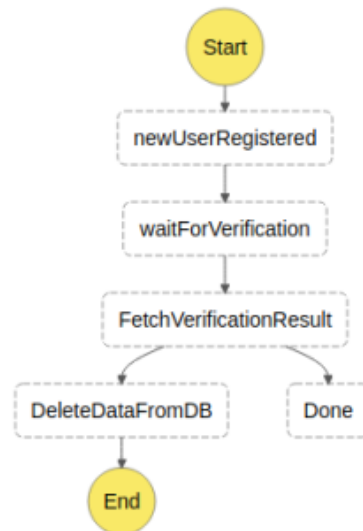


Figure 3.13: Exemplar Serverless workflow, Source: [38]

In the figure 3.13 is an example use-case. Users can register into a web application and their accounts have to be verified. The Step Function starts with the user's registration and the second step is to wait for 30 minutes for the verification. If a user verifies the account, his account will stay in the DynamoDB, otherwise the account will be deleted.

3.6 Conversation bots

Lately we have been witnesses of seeing chatbots over multiple websites. A chatbot can also have a serverless solution and we will discuss its design. This solution does not need a Deep Learning model or Natural Language Processing (NLP), because there is a service, AWS Lex, that allows to create conversational bots by providing several example phrases along with Lambda functions to execute and AWS Lex then creates the NLP model. This can be enriched by using AWS Polly, its purpose is to transform text into speech, so the chatbot can respond in voice [36]. In the figure 3.14 we can see the chatbot pattern.

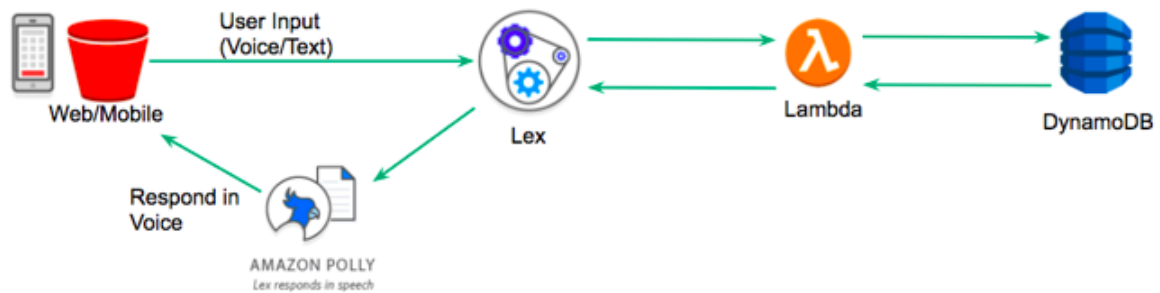


Figure 3.14: Chatbot pattern, Source: [36]

4. Benefits and Drawbacks of Serverless Computing

This chapter discusses the benefits and drawbacks of Serverless computing in a specific context. The benefits are related to eliminating server-side operations, that lead to reduced labour and resource costs, reduced risk, increased flexibility of scaling and shorter lead time. The drawbacks proceed from the Serverless solutions, which can bring a higher latency and loss of control over various aspects. Development of a Serverless application can also bring complications when it comes to the local testing or debugging. A commonly discussed performance issue is Cold Starts, which applies to all the Lambda functions.

As an example, where we can demonstrate certain benefits and limitations can be used a web application, which is suitable for Serverless computing. This application was created by the author of this thesis during an online course. It is a notes application and it allows an user to add, edit, delete, and display notes. The application is written in JavaScript and is composed of various Lambda functions, API Gateway, DynamoDB, and AWS Cognito. The development of such application requires the decision of how to divide the whole functionality into individual functions. In this case we use six Lambda functions, where five of them handle the DynamoDB database (get notes, get note, create note, update note, and delete note), and one function handles the authorization. To test and debug this application, it is necessary to deploy the code on the Lambda functions, because otherwise we could not simulate the environment, which is composed of the https requests by the API Gateway and the interaction with DynamoDB database. If we compare this development with a non-Serverless monolithic application, the non-Serverless application can be tested and debugged locally, which facilitates the development. This aspect can be considered as a limitation of Serverless computing. On the other hand, as soon as the Serverless application is developed and tested, there are no requirements to handle and operate any kind of server. The application has been deployed in September 2019, and even though it has not been used, it is still accessible on the internet and we can still benefit from its functionality and yet we have not been charged for anything. Considering an application with the same functionality, but deployed on a server, we would have to pay a constant amount monthly for running the server, to make the application accessible. We can come to a conclusion, that the costs for compute resources in this case are definitely a benefit. Apart from the costs, we are not responsible for the server and for maintaining the Serverless application, which is also a great benefit. If we imagine, that this application is be used by a large number of users, the Serverless application can automatically scale up to process all the incoming requests. As the application is composed of various functions, we do not need to talk about scaling the whole application, but we can suppose, that only one of the functions will be scaled up and the other functions wont be executed at all. For instance, a teacher assigns a task to the students and they all write a new note about the task to this application. The only function, which will be scaled is "create note" and we will only be charged for executing this particular function. A non-Serverless

application needs to have enough compute resources to do the same thing and probably also an additional configuration, that enables the scalability. In case the non-Serverless application does not have enough compute resources, we have to allocate more resources or even deploy an additional server. The scalability is definitely a positive aspect of the Serverless computing.

It is also important to mention, that not all applications are suitable for Serverless computing. For example, a large application, which has a thousands of requests per seconds, needs to spin up hundreds of instances of Lambda function and the total costs could exceed the costs of an EC2 instance. [39] As an example, we can imagine an online game or any other application, which is basically running constantly.

4.1 Benefits

Serverless computing offers multiple benefits to consider while deciding whether to use Serverless or not. The often mentioned benefits are definitely the costs of Serverless as we only pay for what we are using. That may be true, but the other side of the costs are also the reduced costs during the whole development process, bug fixing or others, because we also save a lot of time to the developers.

4.1.1 Reduced Labor Cost

As we have already mentioned in the first chapter, with Serverless there is no need to take care of the server side of a Serverless applications and developers are free to dedicate more time to the business logic. On top of that there is no need for them to occupy themselves with the additional environmental settings. The provider of the Serverless services will take care of it.

If we look at the architectural patterns, we notice that there is more benefits connected to the reduction of labour costs. The obvious example of this benefit is the authentication. Whether we are building a public application or an internal business application, we will need to manage the authentication. Instead of defining our own code and user management, we can simply implement a tool for authentication.

The software development is also simplified, because we do not need to think about the infrastructural code. An example of that is developing an application using HTTP API Services. API Gateway takes care of the request and response processing.

Deployment with FaaS is easier as well, because we deploy only the code containing the logic. In case of Javascript or Python we upload only source code or a zip file and in case of Java Virtual Machine (JVM) based languages we upload a Java Archive (JAR) file. The execution is ready right after uploading the code, there is no need to think about other configuration

that need to be done in traditional ways, like configuring the Docker.

4.1.2 Reduced Risk

Thinking about the software development and the problems that can occur we find out, that we are very susceptible to many kinds of failures. The more systems, components and teams that are managing them we have, the more problems can occur. By passing the responsibility for managing some of the components of the application to external experts, we avoid the need of fixing those failures by our own teams. This is very important, because if there is a problem that might have a negative impact on our business, we need to solve that situation immediately. That can be difficult to achieve, because our experts need to detect the problem and solve it, which can take a certain amount of time.

It needs to be mentioned, that we are still responsible for multiple elements, especially those, that are related to the logic of the application. This is usually not so problematic, because developers understand it well as they work with the logic layer very frequently. Then it is easier to identify the error and fix it.

For example, if we are managing a NoSQL database. Once, everything is configured, it is not very probable that any kind of failure in a node would occur. But what happens if it does? Even though our team is experienced, we can find it very difficult to identify the exact problem and fix it. The more time spent on handling the error, the more impact it will have on our business. If we choose to use for example AWS DynamoDB, we do not need to even think about this scenario. The experienced team of the provider is dedicated to this service and will quickly handle such situation [1].

4.1.3 Reduced Resource Cost

To operate a traditional application it is necessary to figure out how many and what kind of underlying hosts is optimal for running the application. The hosts may be servers, data servers, Random Access Memory (RAM), Central Processing Unit (CPU), and others. What do we need to support scaling or a high availability? Once we have decided what resources or hosts we are going to use, we need to think about which part of the application will be running on which resource. Finally, we need to acquire the hosts and the resources.

It may sound straightforward, but in reality it is a very complicated process. It would be very rare to know what the resource requirements are and how they are going to change in the future. We need to ensure that the application will never be short on those resources, otherwise it would lead to a failure. On the other hand it is inevitable, that the application will be sitting idle sometimes, so in this case, we do not need any resources. In any case we cannot change the resources according to the current use of the application so we need to provide enough resources to cover the maximum use. The maximum use is not easy to

estimate, so it often happens that we overestimate it. In case the application is sitting idle, we still have to pay for the servers and other resources even if they are not in use.

In case of Serverless application, there is no need to plan, allocate, or provision the resources. The application will only use the necessary amount of resources to run successfully and we do not even have to think about how it works. For example, if our application is running 5 minutes during one hour and 3 minutes during another hour, we only pay for those 8 minutes, not for 60 minutes of each hour. In case we need to store 2GB of data, we only need the capacity of 2GB not 100GB [1].

A study [14] compares costs of running a web application in the cloud using FaaS, microservice and monolithic architecture. For each of the architectures two applications with different parameters of the components are developed. These applications are labeled as S1 and S2. In order to compare the costs three scenarios are defined. The first scenario sends 20% of the requests to the application S1 and 80% to S2. Second scenario divides the requests with 50% and the last scenario allocates 80% requests to S1 and 20% requests to S2. The exception is the monolithic application, which processes 20% of the requests in the first scenario, 50% in the second scenario and 80% in the third scenario. Each of those architectures are deployed in a different infrastructure, and the study defines how to calculate the metrics to easily compare the costs. The result can be seen in the figure 4.1, where the FaaS architecture can reduce more costs than the other architectures.

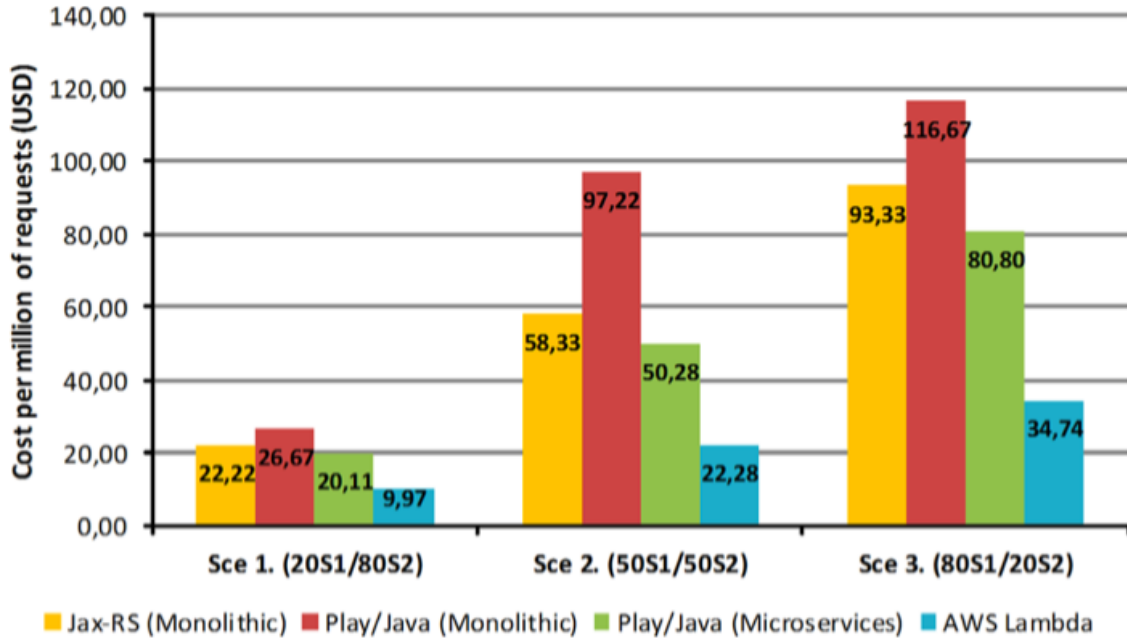


Figure 4.1: Cost comparison per million of requests, Source: [14]

4.1.4 Increased Flexibility of Scaling

Scaling is often used as a huge benefit of Serverless computing. We also indicated scaling in the previously described benefit of reduces source cost. But we did not mention how to achieve that scaling. Do we need to configure any auto-scale groups? Or monitor our application and according to that scale it? We do not need to do anything like that, the scaling is completely automatic.

For instance, AWS Lambda offers concurrent execution, which means that when a Lambda is triggered, AWS creates an instance of this function and runs the execution. If this function is triggered again during the first instance is executing, AWS creates another instance for the execution. This is a process, that happens automatically and there is not need to set anything up [1].

4.1.5 Shorter Lead Time

Considering the benefits mentioned above, we can notice, that they all may save a lot of time. There is a big number of activities that are done automatically without any effort of our developer teams. This leads to the fact, that it is possible to build even a complex application much faster then using traditional ways. The former Cloud Architect at Netflix, Adrian Cockcroft, said the following:

"Small teams of developers are building production-ready applications from scratch in just a few days. They are using short, simple functions and events to glue together robust API-driven data stores and services. The finished applications are already highly available and scalable, high utilization, low cost, and fast to deploy."

We have seen great technologies for software development like Docker, continuous delivery, automated testing and others. Those technologies work very well, but only once they are configured and stabilized. Every time there is a need to extend those resources, there is also a need to reconfigure them. Serverless eliminates all those complexities of building, deploying, and running applications in production, so the time spent on the whole process is significantly lower.

The chief technology officer (CTO) of Comcast Cable, Sree Kotay, said at the AWS Summit in August 2016 the following: *"After going through this journey [of cloud and Agile] for the last five years we've realized benefits, and these benefits are around cost and scale. And they're critical and important, but interestingly they're not the compelling bit...The key part is this really changes your innovation cycle, it fundamentally shifts how you think about product development."* [1]

4.2 Drawbacks

The main benefits of Serverless computing were described in the previous chapter. Those benefits can be a considerable reason for companies or developers to choose Serverless over the traditional way of development. Those reasons are mostly reduced costs, but also reduced time of development or responsibility for the whole product. In this section we will realize, that Serverless brings also many drawbacks, that might make the development process harder. We need to realize, that Serverless is a new different way of software development and some activities may be more difficult or problematic compared to the traditional way.

4.2.1 Latency

As a latency we can understand the time interval between the request and response of the service. In a traditional software development, latency can be reduced by co-locating components on the same host instance or bringing them together in a same process. Within the development of non-Serverless application we can also optimize communication between these components or customize data formats and network protocols. Serverless does not offer those benefits mentioned above. The most common instrument supporting the inter-component communication is HTTP API, which might be slower than other transfer protocols. More complex Serverless application using an un-optimized transfer protocol will lead to higher latency. Another problem affecting the latency is "Cold Starts", which will be discussed in more detail later in this chapter [1].

A study from the Indian Institute of Technology Kharagpur was dedicated to improve latency by caching techniques. The study compared two different approaches of software development. One of them was a simple Serverless database application using API Gateway, Lambda and DynamoDB, and the other one was an application with the same functionality in a virtual machine on AWS EC2, which provides compute capacity and is a shortage for Elastic Compute Cloud. The components used for the second use case were MongoDB database, Python Flask as a web framework, Nginx as a web server and uWSGI as an application server. Both of those applications were deployed in five different regions and the study observed the mean database access latency[15].

As we can see in the figure 4.2, there is a clear difference between both approaches. The Serverless application using DynamoDB has a significantly higher access latency in all regions, in comparison with the virtual machine based application using a database MongoDB. According to the study, the most probable reason is that Serverless components are not hosted on the same physical host.

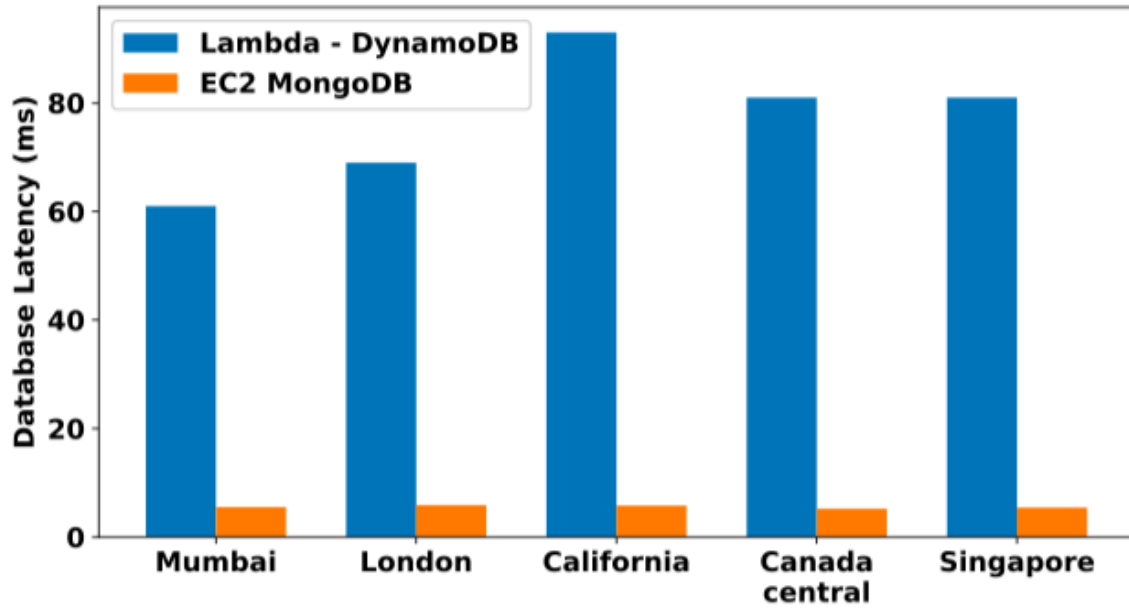


Figure 4.2: Database access latency in Serverless and Virtual Machine (VM) based architecture, Source: [15]

4.2.2 Cold Starts

Cold Starts is a commonly discussed performance issue within Serverless. To run an execution of a Lambda function, the AWS Lambda platform initialize a container, where the code is run. Initializing this container can take a significant amount of time, so the duration of the execution will be higher. The container stays active after the last invocation for a couple of minutes, so all execution run in this container will be faster than the first one. This is applicable for concurrent executions as well.

This can be an issue when our functions are sitting idle for most of the time, because the probability of initializing a new container for each execution is very high and it can have a negative impact on the end-user's experience or the business processes. If this is not the case and the functions are used frequently, the containers will stay warm, so it will not have any negative affect on the duration [1].

To show this, we conducted an experiment, which consists of executing a Lambda function. The function is written in Java 8 and the main logic is to transform the Extended Markup Language (XML) files with Extensible Stylesheet Language Transformations (XSLT). The XSLT imports files and has parameters. All the files are encoded with Base64 and are sent within the HTTP API in the body. We made 300 executions and observed their time duration.

The figure 4.3 displays a plot generated by the AWS CloudWatch and we can clearly see the difference between the first execution and the following ones. Excluding the very first execution we observed, that the following three executions were still quite slow compared to the other ones. The maximum execution time according to this plot is 11600ms, however

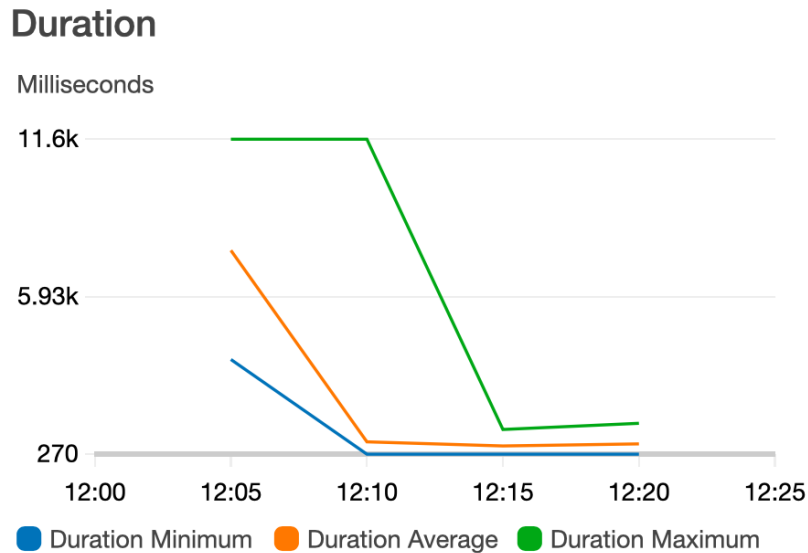


Figure 4.3: Cold starts, Source: Author

the total time from request to response was 12890ms, which is a significant difference when compared with the minimum execution time 270ms and the total time 570ms.

4.2.3 State

One of the main characteristics of Serverless computing is that most of its components are stateless. That means, that those components only implement logic and execute parts of code, but they are not able to store data in the memory for later use. This can be an advantage in many particular use cases, but if we need to work with any data or objects, we will need to use a storage where one function uploads data and the next function loads them for further operations. The necessity of one stateless component interacting with a stateful component such as database leads to a higher latency mentioned earlier. This might also be a problem for experienced developers, who are beginning with Serverless development, because it is quite dissimilar to the traditional development [1].

4.2.4 Local Testing

Local testing is a significant limitation while developing. Non-Serverless world allows us to use local components for testing and these components will be probably integrated for deployment in production. In Serverless world we can also execute tests, especially unit tests, but the end-to-end testing of the application is naturally more difficult as we do not really know how exactly the components communicate and what is going on behind our implemented logic. We can classify the difficulties in local testing in two ways. First way is that the infrastructure is inside the Serverless platform so it might be difficult to connect all the components of the application in a realistic way. Secondly, the application consists of many separate pieces and managing all of them can prove to be challenging [1].

4.2.5 Loss of Control

Using Serverless involves a use of third party's operations, configurations and software for the development of our applications. For this reason developers are no longer in control of the whole environment, which can lead to many difficulties and limitations. Those difficulties are related to loss of control over configuration, performance, resolution or security. We will discuss each of these in the following subsections [1].

Configuration

Even though there is a possibility to configure particular parameters, the number of those parameter is limited. For instance, AWS Lambda allows configuration of parameters such as run-time, environment variables or permissions, there is still no control over operating system parameters or Java Virtual Machine (JVM). Other platforms that are providing certain logic, such as AWS Cognito or AWS CloudWatch also expose configuration parameters, but those may be limited and developers do not know anything about how the underlying software is configured and cannot change it [1].

Performance

Just like loss of control over configuration, there is a similar issue with the performance. Loss of control over performance can be classified in two main categories, one is related with the application code and the other one with the underlying Serverless platforms. Serverless underlying platforms conceal details of executions, what are caused by using multiple layers of virtualization to allow an efficient use of the physical hardware.

The book "What is Serverless?" written by Mark Roberts claims the following: *"In observations of benchmarking code running on the AWS Lambda platform, we see that identically configured Lambdas can have drastically different performance characteristics. Those characteristics can vary for the same Lambda over the course of minutes or hours, as the underlying platform alters scheduling priorities and resource allocations in response to demand."* [1] This is quite interesting, because during testing the lambda function to prove the Cold Starts issue we noticed that the duration was varying. When calling the Lambda function throw API Gateway, we had results varying from 2120 ms to 560 ms. Two seconds and a half second is a pretty significant difference and we could suppose, that if a mean duration of a Lambda function would be 2 minutes, the time of execution can vary between 1 minute to 4 minutes. To prove this we created two AWS Lambda functions in different regions and ran multiple executions. The code was written in JavaScript and the task was to identify the prime numbers. The mean time of run executions was about 14300 milliseconds and the surprising result was, that this number did not dramatically vary. The differences between duration could be caused by the time needed to run the code as such. You can see the duration in the figure 4.4. This is definitely not a sufficient prove, we would need to do more experiments to find

out whether it is true or not. AWS providers also keep optimizing their services, which can have a positive impact on performance [1].

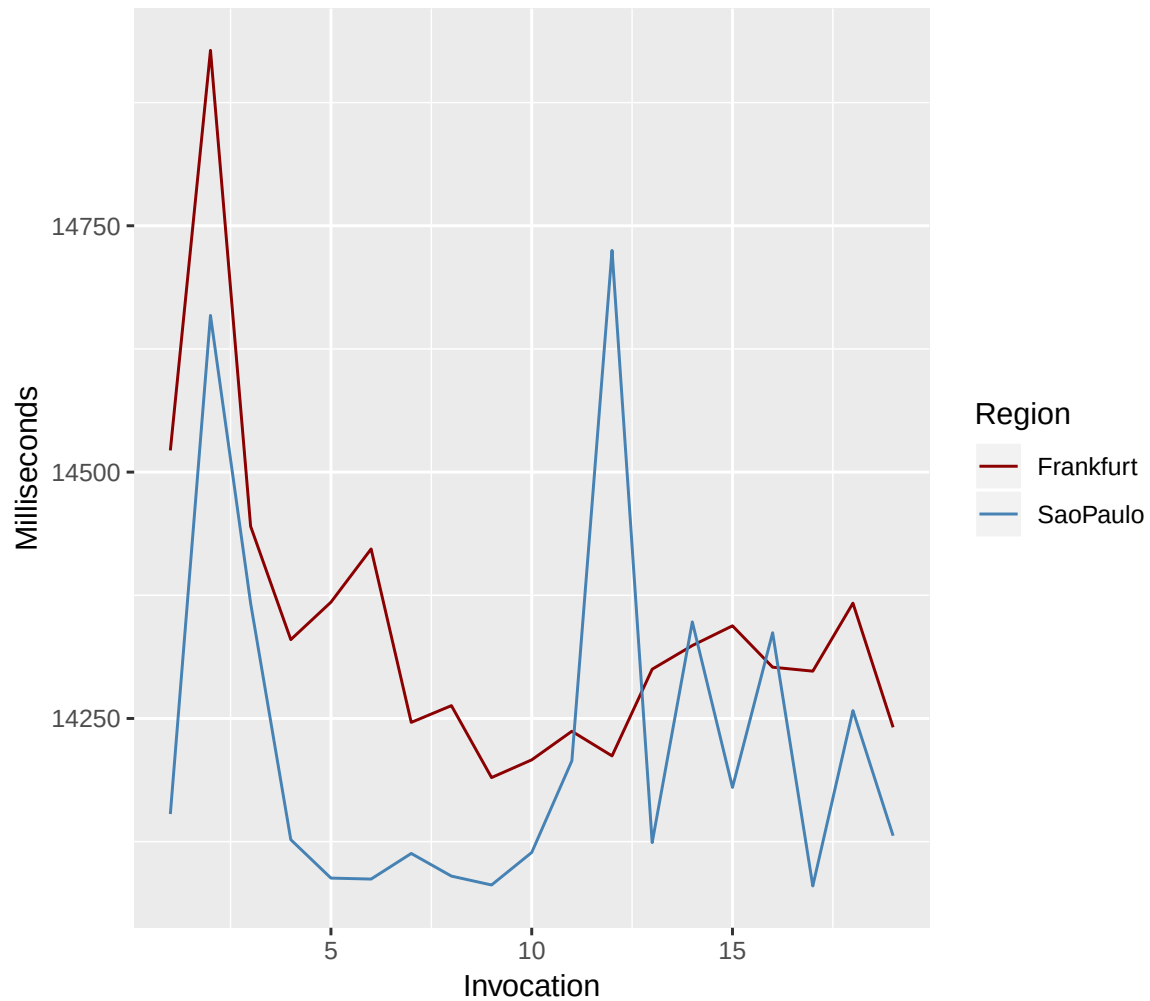


Figure 4.4: Duration of Lambda executions, Source: Author

Resolution

In a traditional development where we can fully control our system, we can take a particular action to solve the problem, whether it is related to a hardware component or an operating system issue. We can even choose when to do the maintenance window or extend the infrastructure. In Serverless we can only resolve issues with the application code or configuration of the services we use. All other issues are in hands of the provider and we do not know whether there is a problem or not. The only possible way to be informed, is to check the AWS status page, where we mostly see a green check with an informative text "Service is operating normally". The problem could occur when there is, for instance, a catastrophic event such as an earthquake that destroys the data center [1].

Security

The only way we can affect the security is via the mechanisms and configuration provisioned by the provider or our own code. Each Serverless platform has its own security features and we are forced to rely on them. In some cases these features are insufficient. This case can be, for example, the API Gateway, which is accessible from anywhere on the internet, which can be an issue for the internal business applications. To make the API accessible only from particular IP addresses we either cannot use this method, or we need to create an authorizer [1].

4.2.6 Debugging

Debugging a Serverless application can be very difficult, because of the loss of control over many aspects. Debugging code in an Integrated Development Environment (IDE) is not that difficult compared to the Serverless function, because there is a possibility to see the execution line by line. In case of a Lambda function there is only the CloudWatch, where we can see only logs of the execution and they may not be very clear. The code as such is possible to debug right in the IDE, but we need to simulate the whole environment, which is not always possible. Furthermore, in case of using JVM, we have to upload a jar file every time there is a change in the code to see if the bug was resolved or not and that can be time consuming [1].

A study dedicated to building a Serverless chatbot [16] considers programming and debugging aspects and comes to a conclusion, that developing functions is straightforward, but integrating those functions into one workflow is a difficult task. According to this study there is a lack of debugging tools in case that the functions fail and it would be convenient to provide access to debugging tools, which could be breakpoints. The study finds the logs, that are not able to report what exactly happened, insufficient.

5. Prototype Application

The prototype application is an event-driven application, that can process various types of documents. In this section we will describe its functionality and afterwards we will outline the technology used for this application, which includes the AWS Services and programming languages. Afterwards we will move to the implementation, which contains a detailed description of the key components of the application and their implementation. Last section focuses on the monitoring and evaluation.

5.1 Description of the Functionality

The main functionality of this application is to transform the XML files into another XML using XSLT. XML is a markup language, its purpose is to store and transport data and it can be readable either by human or a computer. Within XML we define elements, which contain a value. XSLT serves as a language for transforming these files into various types of format. This functionality is used in Electronic Data Exchange, which is the computer-to-computer exchange of business documents in a standard electronic format between the business partners.[40] There are various kinds of EDI documents like invoices, orders, shipping status, and others. EDI has its specific standard format, but as this format has been evolving across the world, there is more than one of them. In case there are two companies using different format, their systems would not understand each other. For instance, a format EDI X12 and ediFACT have the same purpose, but both of them have a different structure and terminology. The EDI X12 is mostly used in the USA, whereas the ediFACT is more frequently used by European companies. Therefore, it is necessary to translate the EDI X12 into ediFACT and the other way round, so both systems of the companies can understand and process these documents[41]. Both of these formats can be transformed into XML format and use the XSLT stylesheet, which will transform these files to the format of the other standard. This is the purpose of the application. This application is able to transform EDI standard format into a XML file, which is then transformed with the XSLT stylesheet into another format.

This application has to provide a storage, where all these types of files (EDI, XML, XSLT) including already processed files can be stored. We can provide also other formats, however, they will lead to an immediate failure of the process. Processing a file has to start immediately when a new file appears in the source bucket. EDI file is firstly transformed into the XML and this XML is then transformed into another XML. We can also provide the XML directly, so the application skips the step of transforming the EDI into XML and transforms only this given XML into another XML. The XSLT is only stored, so it can be used for transforming the XML documents.

5.2 Description of the Technology Environment

The description of the functionality corresponds to an event driven file processing. Depending on the type of a file, the application has to decide, how to process the file. The ideal solution, in this case, is using an Event-driven data processing pattern with the Step Functions. Step Functions are also a key component of building a Serverless workflow, which is convenient for this use case. The execution of the workflow is invoked by uploading the file into a source folder. Depending on the file, the workflow chooses the right steps to handle the file correctly. The steps are mostly Lambda functions, that are triggered by the Step Functions. A step can be also a component "Activity Worker," which supports hosting particular components on a non- Serverless platform. Unlike Lambda function, activity worker runs constantly and processes the incoming requests. The activity worker was an exciting concept that could be used for future applications and processes and we try to explore this solution. The non-Serverless platform is a cloud service EC2 by AWS providing compute capacity.

As a storage is used AWS S3. Within S3 can be created multiple folders, that are usually called buckets and for this use case we use more of them. One bucket serves as a source bucket and invokes the Step Functions. Step Functions sort the types of files, which are then copied to three different buckets. The processed files are also stored in a corresponding bucket.

Lambda functions along with activity worker contain code, which is written either in JavaScript or Java. JavaScript is used for simple Lambda functions, its main purpose is to handle S3 buckets or invoking the Step Functions. On the other hand, Java serves for the transformations from EDI to XML and XML to XML. For EDI to XML transformation is used BerryWorks software [42], which provides packages for Java, that transforms EDI files to XML using one single function. The function, that transforms XML to XML uses a package by Saxonica [43], which is an XSLT processor, that allows this functionality.

5.3 Implementation

In this section we discuss the implementation of the key components in detail. This also involves explication of the essential code. The whole code can be seen in the appendix A. Source Code of the Prototype Application. One of the important aspects is invoking Step Functions with Lambda, which enables to invoke the workflow. We also discuss building the workflow and each step in more detail. The application uses S3 buckets as a storage and it is important to discuss how to handle the files on the S3 buckets. Then we explain in more detail the function, which transforms the XML files into another XML. One of the last topics are activity workers, where we discuss launching of the EC2 in more detail as well as the functionality of the code running on the EC2. The last aspect will be error handling.

5.3.1 Invoking Step Functions

Before we start, it is important to define a region. Amazon Web Services has 18 different regions all around the world as their server rooms and data centres. In Europe it is Frankfurt, Ireland, London, Paris and Stockholm. For each service it needs to be chosen one of those regions. Connected services like Lambda, DynamoDB, StepFunctions always need to share the same region, otherwise they would not be visible to each other. We are using Frankfurt, which is recognized as eu-central-1. The following code is contained in all the functions written in JavaScript:

```
1 AWS.config.update({region: 'eu-central-1'});
```

To invoke the workflow it is necessary to have S3 bucket, which will create an event as soon as a new file is uploaded to this bucket. This is configured in the S3 console in *Properties -> Events -> Add notification*. The type of event is PUT and the event is sent to a corresponding Lambda function containing the following code:

```
1 exports.handler = async (event) => {
2   let uploadedFiles = event.Records.map(async(record) => {
3     let params = {
4       stateMachineArn: process.env.STATE_MACHINE_ARN,
5       input: JSON.stringify(record)
6     };
7     let data = await stepFunction.startExecution(params).promise();
8     return data;
9   });
10  let results = await Promise.all(uploadedFiles);
11  return results;
12 };
```

This function receives the event and processes the records. Depending on the number of uploaded files there will be a different number of records. These records will be mapped as parameters and for each parameter one State Machine's workflow will be invoked. We have declared an environment variable, which is *STATE_MACHINE_ARN* and contains the ARN (Amazon Resource Name) of the Step Function, that will be executed. The function also needs a permission to be invoked by the S3 bucket event. Permissions are required in most of the AWS services that are used for this application.

5.3.2 Building a Workflow

AWS introduced the ASL (Amazon States Language) for defining the Step Functions workflows. The ASL takes over the JSON format and enables us to define the particular states over the workflow. Step Functions are endowed with multiple types, which determine the

purpose of the particular state. In this case, we will mostly use the task type, which requires a resource that executes a given function or an activity worker. Apart from that, we will also use a choice type, and its purpose is to evaluate inputs, and based on that, it will choose the following step. The last used type is a fail, which leads to a complete failure of the state machine. The following code demonstrates a sample of the workflow definition:

```

1  {
2    "StartAt": "GetFileType",
3    "States": {
4      "GetFileType":{
5        "Type": "Task",
6        "Resource": "arn:aws:lambda:eu-central-1:630997192154:function:getFileType",
7        "ResultPath": "$.results.fileType",
8        "Next" : "CheckFileType",
9        "Catch": [
10         {
11           "ErrorEquals": ["States.ALL"],
12           "Next": "Failure"}}]}

```

AWS generates a graphical visualization of workflows that facilitate the understanding of the process and can lead to effortless monitoring. In the figure 5.1 is the visualization generated by AWS:

As can be seen in the workflow, it is composed of various steps. Each step has a different functionality which we describe below:

- **GetFileType:** A Lambda function takes as an input the name of the file and splits the name and type of the file. This type will then be an input into the following step. In a case of failure, the following step will be "Failure", which stops the whole execution.
- **CheckFileType:** This step is of type "Choice", where the step evaluates the input, which is the type of the file. If the type does not correspond to any established type, it leads to a failure as in the previous case. Otherwise it will choose another step depending on the type. Allowed types are XSLT, XML and EDI.
- **Failure:** This step's type is "Fail" and it causes a termination of the whole process.
- **UploadEDIToS3:** This is a Lambda function, which will place the file into the corresponding S3 bucket.
- **EDIToXML:** The EDI files need to be transformed into xml format to enable the transformation with the xslt file. In this case we use an activity worker, which is running on an EC2 instance. The EC2 is not Serverless, but the activity worker allows us to run a particular functionality on a non-Serverless platform.
- **UploadXMLtoS3:** As in the previous case we put a file of type XML to a corresponding S3 bucket.
- **GetXSLTmap:** This workflow has to satisfy various types of EDI or XML documents, whereas each type of document is unique even within partners. For this reason we need a specific xsl file for each xml to achieve a correct transformation. This requires to have a code, which will go through the xml and get the correct xslt file. This functionality

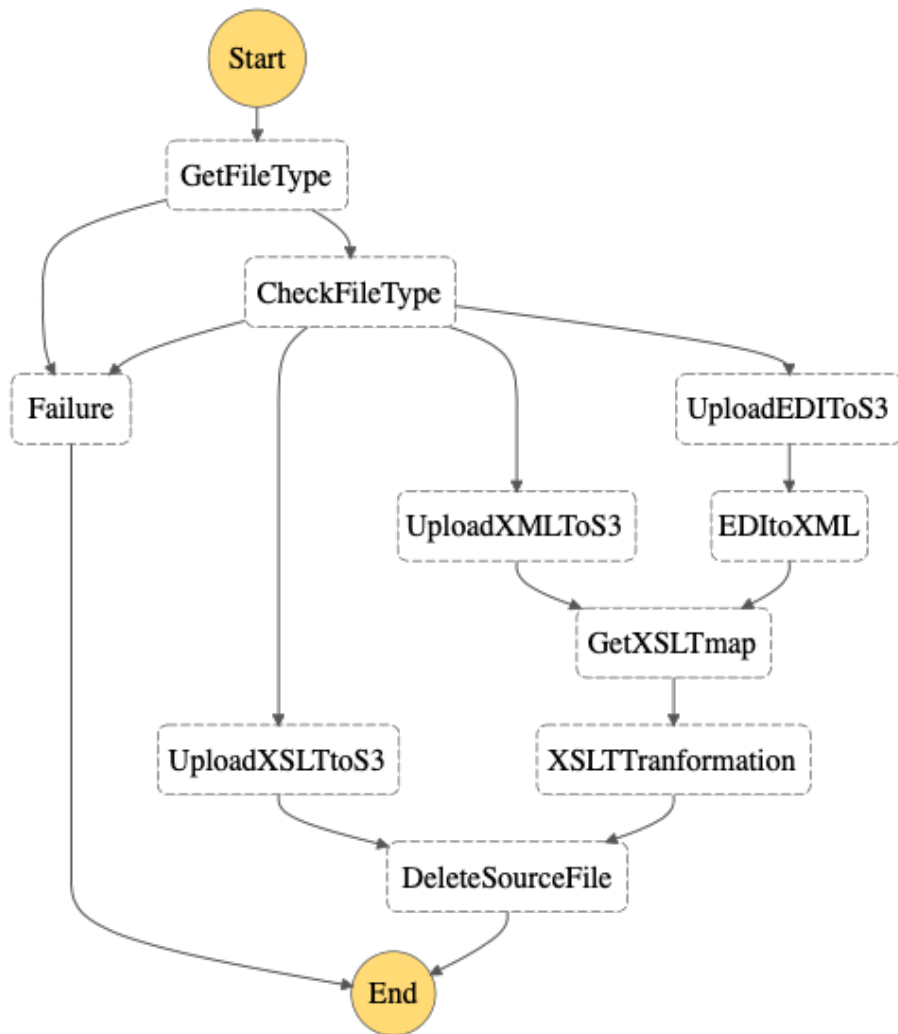


Figure 5.1: Workflow of the prototype application, Source: Author

is too complex to write and is not the key component for this prototype application. That is why we use a default XML files, that will be able to transform any kind of file.

- **XSLTTransformation:** This is a Lambda function using Java 8 runtime. The function takes a name of the XML file as an input and runs the XSLT transformation using default XSLT file.
- **UploadXSLTToS3:** A Lambda function again puts the XSLT file into a specific S3 bucket.
- **DeleteSourceFile:** If the previous steps are successful, a function deletes a source file from the S3 bucket.

5.3.3 Operating S3 Buckets

For the purpose of operating the S3 buckets are mostly used Lambda functions written in JavaScript and in some cases also in Java. JavaScript is compared to Java easier to deploy, because we do not need to provide a JAR file as in case of Java. JAR file is a Java Archive, which aggregates all the code, metadata and resources and it usually takes a couple of seconds to create. The code bellow is implemented in a Lambda function, which is executed in a step called *UploadXSLTtoS3*. The functionality of this example is to copy the XSLT file from a source bucket, to a bucket containing only XSLT files. The *"event"* is an input into this Lambda function in a JSON format. From the input we can obtain a key, which is a name of the file, using *"event.s3.object.key"* and a name of the source bucket using *"event.s3.bucket.name"*. For copying the file one more parameter is necessary, it is the destination bucket. Destination bucket is an environment variable, that can be added manually in the AWS Lambda console. The environment variable can be accessed using *"process.env.DESTINATION_BUCKET"*. Given the name of the file, source bucket and destination bucket, we can call a method *"s3.copyObject(params)"*, which copies the file. The code is as follows:

```
1 // UploadXSLTToS3 - Copy XSLT to S3
2 exports.handler = async (event) => {
3   let params = {
4     Bucket: process.env.DESTINATION_BUCKET,
5     CopySource: encodeURI('/'+ event.s3.bucket.name + '/' + event.s3.object.key),
6     Key: event.s3.object.key
7   };
8   await s3.copyObject(params).promise();
9 };
```

The other example of handling the files is written in Java and its purpose is to read the XML file on an S3 bucket and save it in a variable *"file"* to allow further processing. For this task it is necessary to provide a source bucket and the name of the file. Java will read the file line by line and store it in the variable. This function is the following:

```
1 /*Get XML file from S3 bucket*/
2 public byte[] getXmlFile(String fileName) {
```

```

3      S3Object s3Object = s3Client.getObject(new GetObjectRequest(SOURCEBUCKETXML, fileName));
4      InputStream objectData = s3Object.getObjectContent();
5      Scanner scanner = new Scanner(objectData);
6      String file = "";
7      while (scanner.hasNext()) {
8          file += scanner.nextLine();
9      }
10 }

```

5.3.4 Transforming XML Files on AWS Lambda Platform

The following code contains the logic of the XSLT transformation. This function takes as an input XSLT and XML files and runs the transformation. For the transformation we need an instance of *"TransformerFactory"*, which allows us to declare a *"Transformer"*. This transformer needs the XSLT as a parameter and it will compile it. If the XSLT file is invalid, the function will fail. If it is successful, it will call the function *"transformer.transform(xmlFile, result)"*, which needs as a parameter XML file and a result. The result is an output of this function. If the transformation is successful, it will return the transformed file. The code in Java is as follows:

```

1 public String transform(byte[] XSLT, byte[] XML) throws TransformerException {
2     TransformerFactory tFactory = TransformerFactory.newInstance();
3     final StringWriter writer = new StringWriter();
4     Transformer transformer = tFactory.newTransformer(XSLT);
5     StreamResult standardResult = new StreamResult(writer);
6     transformer.transform(XML, standardResult);
7     return writer.toString();
8 }

```

5.3.5 Activity Workers

Activities are defined in the AWS Step Functions as it follows: *"Activities are an AWS Step Functions feature that enables you to have a task in your state machine where the work is performed by a worker that can be hosted on Amazon Elastic Compute Cloud (Amazon EC2), Amazon Elastic Container Service (Amazon ECS), mobile devices—basically anywhere."*^[19] This application uses the activity worker, which transforms the EDI messages into XML structure. It is written in java and running on an EC2 instance. We will firstly launch the EC2 instance to which we need to attach a Identity and Access Management (IAM) role. The IAM role contains permissions for the particular server, which will allow us to access our S3 buckets. The AWS generates a ssh key, which we have to provide for connecting to the EC2 instance. The command to connect to the server is the following:

```
ssh -i "key.pem" ubuntu@ec2-52-57-251-64.eu-central-1.compute.amazonaws.com
```

To compile and run java code we need to install the Java Runtime Environment. This can be done with the following command:

```
sudo apt install default-jre
```

Now we can copy a jar file to the server. We will discuss below how we have created it.

```
scp filename.jar ubuntu@ec2-52-57-251-64.eu-central-1.compute.amazonaws.com:~/java
```

For the purpose of this application we are not considering any further configuration of this server, nor running the application issues. That is why we run this component with the following command:

```
java -jar filename.jar
```

Within the Step Functions console, we create an activity. The activity does not need any specific configuration, the only element we need to fill is a name of the activity. When the activity is created, the AWS provides us with the Amazon Resource Name (ARN), which we will have to define in our code.

The JAR file contains some important parts, that we will mention below. Firstly we will mention a function, which enables processing the request sent from the workflow. Before defining the function, we need to provide the ARN of the activity and credentials. To obtain these credentials we will create a user within the IAM console. Having the credentials, which are the access key and the secret key, we can either set them as system variables or put them directly into the code. If we put them directly into the code, the code is as follows:

```
1      /*Credentials and ARN*/
2      private static String ARN = "arn:aws:states:eu-central-1:630997192154:activity:editoXML";
3      private static String ACCESS_KEY = "AKIAZF2S5BXNJ5JOS2UO";
4      private static String SECRET_KEY = "*****";
5      private BasicAWSCredentials credentials = new BasicAWSCredentials(ACCESS_KEY, SECRET_KEY);
```

With the following code we define the AWS Step Functions. We need to provide region and credentials and then our client can be built. The function *"listen()"* consumes the requests sent by the Step Functions and as a request is sent, it will start processing the input. The input is in JSON format, so we need to extract the name of the file. A function that we mentioned earlier will read the EDI file from the S3 bucket, transform this file and upload the file to the final bucket. To transform the EDI file and upload it to the S3 bucket we use the following method: *"s3.upload(new editoXMLprocessor().transform(EDI), fileName)"*. If the file is processed successfully, the activity worker will send task success request and the

processed file will upload to S3. Otherwise the activity worker sends an fail task and the state machine will be either terminated or retried, depending on the workflow definition. The code is as follows:

```
1     public void listen() {
2         final AWSStepFunctions client = AWSStepFunctionsClientBuilder
3             .standard()
4             .withRegion("eu-central-1")
5             .withCredentials(new AWSStaticCredentialsProvider(credentials))
6             .build();
7
8         while (true) {
9             GetActivityTaskResult getActivityTaskResult = client.getActivityTask(
10                 new GetActivityTaskRequest().withActivityArn(ARN));
11
12             if (getActivityTaskResult != null) {
13                 try {
14                     final JsonNode json = Jackson.jsonNodeOf(getActivityTaskResult.getInput());
15                     if (json != null) {
16                         if (json.get("key") != null) {
17                             fileName = json.get("key").textValue();
18                             S3processor s3 = new S3processor();
19                             EDI = s3.get(fileName);
20                             s3.upload(new editoXMLprocessor().transform(EDI), fileName);
21                             result = process(fileName);
22                             client.sendTaskSuccess(new SendTaskSuccessRequest()
23                                 .withOutput(result));
24                         }
25                     }
26                 } catch (Exception e) {
27                     client.sendTaskFailure(new SendTaskFailureRequest());
28                 }
29             }
30         }
31     }
```

For EDI to XML transformation we explored berryworks software, which offers a very simple solution. The following function takes the EDI file as an input and returns XML. The code is as follows:

```
1     public String transform(String EDI){
2         reader = new StringReader(EDI);
3         writer = new StringWriter();
4         new editoXML(reader,writer).run();
5         return writer.toString();
6     }
```

5.3.6 Error Handling

Error handling here is used especially for the first two steps, because if these steps are not successful, they automatically lead to failure. Within Step Functions we can handle many

various kind of errors and react on them. In this case any kind of error leads to a failure of the executions, which is defined directly in the workflow using the following code:

```
1 "Catch": [
2     {
3         "ErrorEquals": ["States.ALL"],
4         "Next": "Failure"
5     }
6 ]
```

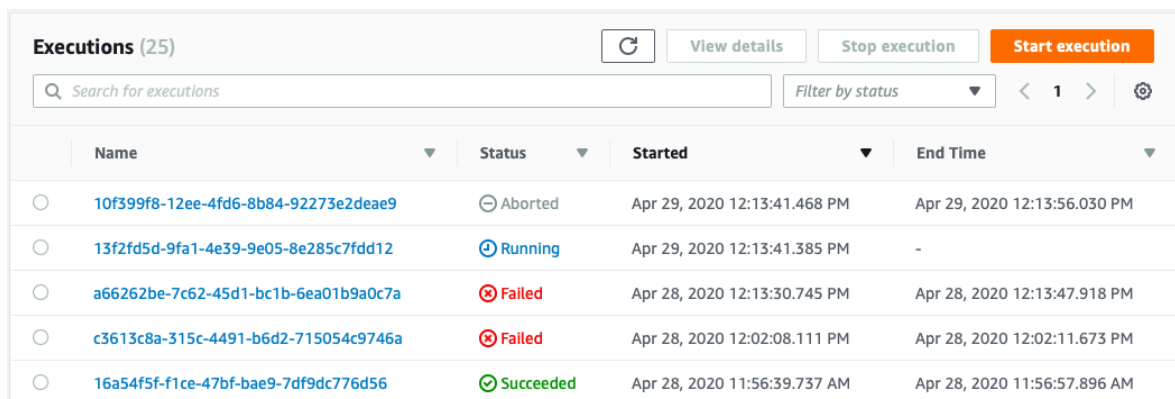
It is also possible to retry a specific step multiple times or to react to various kind of error differently.

5.4 Monitoring and Evaluation

In this section we will discuss how we can monitor the executions of the Step Functions and evaluate their various aspects.

5.4.1 Monitoring

The step functions enable a detailed monitoring and it is possible to see each execution of the workflow. In the figure 5.2 we can see the console that allows us to monitor. The first column is a name of the instance, one disadvantage may be that we do not know what file is it exactly. Next column is a status of the the execution. There are four types of status - Succeeded, Failed, Running and Aborted. If there is a execution, which is running, we can choose to stop it, which will lead to the status Aborted. Also, we can see the start of the execution and end time.



Executions (25)				
Search for executions Filter by status Refresh View details Stop execution Start execution				
	Name	Status	Started	End Time
☐	10f399f8-12ee-4fd6-8b84-92273e2deae9	⊖ Aborted	Apr 29, 2020 12:13:41.468 PM	Apr 29, 2020 12:13:56.030 PM
☐	13f2fd5d-9fa1-4e39-9e05-8e285c7fdd12	⌚ Running	Apr 29, 2020 12:13:41.385 PM	-
☐	a66262be-7c62-45d1-bc1b-6ea01b9a0c7a	⊗ Failed	Apr 28, 2020 12:13:30.745 PM	Apr 28, 2020 12:13:47.918 PM
☐	c3613c8a-315c-4491-b6d2-715054c9746a	⊗ Failed	Apr 28, 2020 12:02:08.111 PM	Apr 28, 2020 12:02:11.673 PM
☐	16a54f5f-f1ce-47bf-bae9-7df9dc776d56	⊙ Succeeded	Apr 28, 2020 11:56:39.737 AM	Apr 28, 2020 11:56:57.896 AM

Figure 5.2: Step Functions monitoring, Source: Author

To get more detailed information about the specific execution, we can click on the execution name. Figures 5.3 and 5.4 display the execution details. In the visual workflow we can see

the exact step, which is still in progress. In this case the reason why it is in progress is, that the activity worker is not running. We can also see the input and output of the step, which is in JavaScript Object Notation (JSON) format. In case that the step fails, the step would have a red color.

In the execution event history we can see in detail, what step was executed and its duration. In case of using the Lambda function, we can go to the CloudWatch logs, which exactly logs the execution of the function.

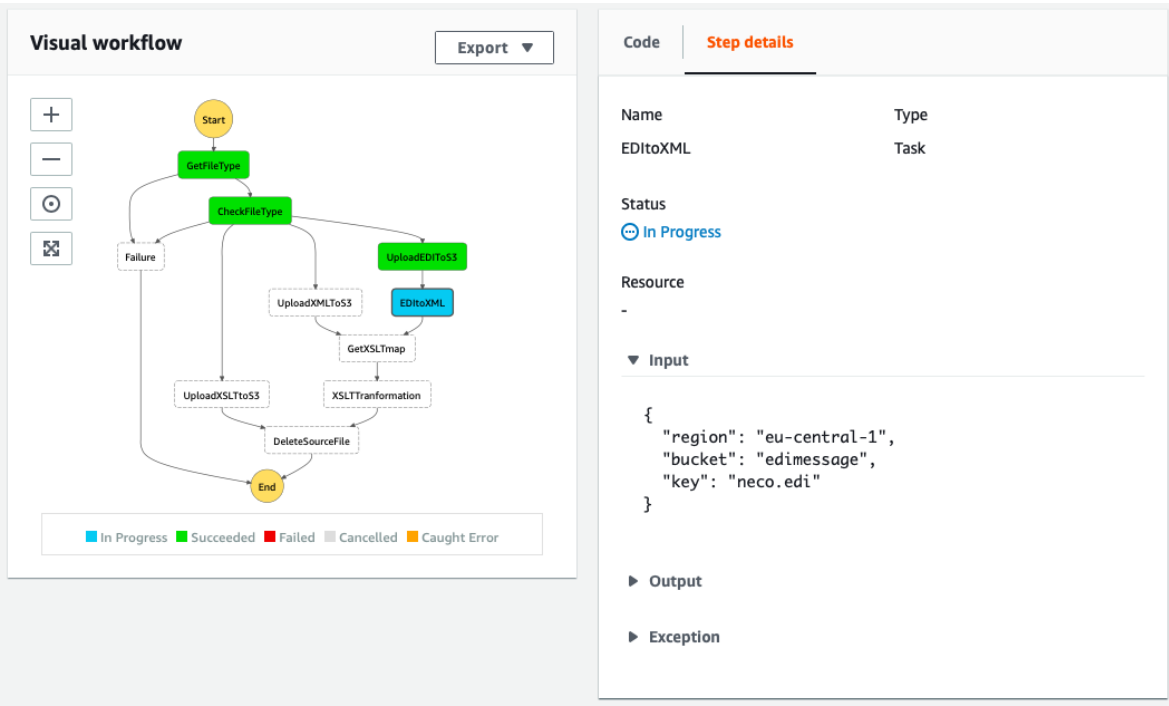


Figure 5.3: Execution details with visual workflow, Source: Author

5.4.2 Overall evaluation

We have created a workflow, that can store the XSLT files, transform the files from EDI format to xml, and transform xml into another xml using the xslt file. It was confirmed, that it is possible to run the xslt tranformation on AWS Lambda platform. For the EDI to xml transformation we have explored berryworks software, which offers a very simple solution. Also we have confirmed that it is possible to use a non-Serverless components within a Serverless workflow. In the following subsections the impression of the development on the author will be discussed.

Entry Barrier

I personally learned to work with some of the AWS services within a few weeks, which is quite a limited window of time. I believe that more experienced developers can learn to work

Execution event history					
ID	Type	Step	Resource	Elapsed Time (ms)	Timestamp
▶ 1	ExecutionStarted		-	0	Apr 29, 2020 12:13:41.385 PM
▶ 2	TaskStateEntered	GetFileType	-	27	Apr 29, 2020 12:13:41.412 PM
▶ 3	LambdaFunctionScheduled	GetFileType	Lambda CloudWatch logs	27	Apr 29, 2020 12:13:41.412 PM
▶ 4	LambdaFunctionStarted	GetFileType	Lambda CloudWatch logs	60	Apr 29, 2020 12:13:41.445 PM
▶ 5	LambdaFunctionSucceeded	GetFileType	Lambda CloudWatch logs	369	Apr 29, 2020 12:13:41.754 PM
▶ 6	TaskStateExited	GetFileType	-	369	Apr 29, 2020 12:13:41.754 PM
▶ 7	ChoiceStateEntered	CheckFileType	-	376	Apr 29, 2020 12:13:41.761 PM
▶ 8	ChoiceStateExited	CheckFileType	-	376	Apr 29, 2020 12:13:41.761 PM
▶ 9	TaskStateEntered	UploadEDIToS3	-	477	Apr 29, 2020 12:13:41.862 PM

Figure 5.4: Execution event history, Source: Author

with the AWS much faster. The AWS also provides a very extended documentation with the use cases.

Comparing Languages

From my point of view it is easier to write the Lambda functions in JavaScript because it is less difficult to deploy the function. Unlike Java, for which we have to provide a jar file, JavaScript can be edited directly in a AWS console. Creating the jar files may have some issues and for this specific case was necessary to create a shaded jar and use some additional maven dependencies. Without the shaded jar it was impossible to run the function on the AWS platform, even if it worked well in the IDE and locally. It was also much easier to test and debug the JavaScript function. Debugging usually consists of finding the error, correct it and running the function again. As we can edit JavaScript in the AWS console, it is not very difficult to save the code and run it again. In case of java, we need to build a jar file, that has to be uploaded to the AWS console and when it is saved, we can run it again. Creating the jar file takes 14 seconds in this case and uploading the code to the AWS Lambda console takes about 55 seconds, so debugging the Lambda function written in Java can take even more time than usually. Also we can see a difference between Java and JavaScript while processing json, that is always an in most of our cases the input and also the output of the functions. JavaScript uses a simple code, where we can get an element within one line of code and it is also not hard to find this solution on multiple sources like stackoverflow. Java provides various solutions that usually need more than one line and in our case we tried multiple solutions, where some of them worked in the IDE, but did not work later when deployed on the AWS Lambda platform.

Workflow

This workflow was inspired by Riyaz Sayyad, who has an online course on udemy. He uses a workflow for processing files as well and he uploads different processed as well as unprocessed files to an S3 within the workflow as it is in this case. My opinion is that in case of the EDI and XML files it is not necessary to have these files on an individual S3 bucket. I think it can raise the total costs for executing the workflow, but on the other hand it can be useful for monitoring this application. If there are hundred of files and some of them fail, they will remain in the source bucket, which makes it easier to detect these files. In case we need to look at the files that are already processed, we can go to the corresponding bucket and find them.

Use of this Application

A purpose of creating this application was to explore the Step Functions and the Lambda functions. It wont be used for any further process, however, some of the parts of this application are exciting. What is especially exciting is the idea of having a workflow containing the state, using Lambda for transforming the xml files and the activity worker, which allows us to have a hybrid application, where some components are Serverless and some are not. We could have a discussion about the difference of using workflow, or using simply Lambda functions, that may be triggered by another Lambda function, which is possible according to the AWS documentation [20]. To find out, which approach is cheaper and has a better performance, we would need to test both of the applications.

Comparing the Application to a non-Serverless Monolithic Solution

In our Serverless prototype application we use a non-Serverless component EC2, where we deploy a simple program, that transforms EDI to XML. This program is written in Java and the only requirement is to install a Java Runtime Environment to the server, which makes it possible to compile and run the Java code. The rest of that application is composed of Serverless components, which require only choosing a programming language for the Lambda functions and adding permissions to enable the communication between the components. As an alternative the application can be implemented as a monolithic Java application and deployed on a server, which requires multiple server-side operations. For instance, the server needs to be secured to defeat all the incoming attacks. Also we need to consider issues related to deploying the application and its maintenance. Serverless solution eliminates these server-side operations, so developers can focus only on the logic of the application. Eliminating the server-side operations reduces the responsibility and risk, because the care of the server is taken by the providers.

The scalability of a non-Serverless monolithic application is limited by the size of the server or the resources allocated for the application. Too many requests can exceed the limit, which

can affect the end-user, because the application slows down or the server stops responding. This problem can resolve either adding more server or allocating more compute power to the application. For instance, if the application is deployed on an EC2 instance and the compute resources are insufficient, we have to deploy one more EC2 instance. On the other hand, the server is always on and the application is always running, even if there are currently no requests. Our Serverless application's scalability depends on load and it can be concurrently processed a large number of requests. As the scalability depends on load, the application won't be running if there are no requests.

The previous aspect has an impact on costs, because in case of the Serverless prototype application we pay only for the usage and in case of the non-Serverless application we pay constantly the same amount. To find out which solution is cheaper would be necessary to test both applications. The disadvantage of the Serverless solution is that the costs for running the application are unpredictable, because they depend on the number of executions of the Step Functions and Lambda and in case, there is too many requests, the total costs can exceed the budget for running this application.

Creating a monolithic non-Serverless application with the same functionality needs a different architecture. All the functionality is written within one project written in Java, which connects to a database or a different storage. The benefit of creating a monolithic application is that there is no need to integrate all the components. This facilitates debugging, because we can use the debugging tools provided by integrated development environment (IDE) and the written code is final. Final means, that the code already cooperates with other functions and the deployment won't affect it. In case of developing Lambda functions for the Serverless solution, there is need to test, whether each function processes the input correctly and gives a correct output. Sometimes it is not possible to predict the structure of the input and the code can not process it, so even if the code is working well in the IDE with a certain test environment, it fails when deployed on the Lambda platform. The debugging related to AWS services issues is also problematic, because the logs provided by AWS are compared to the IDE usually insufficient, what we have seen while developing our Lambda functions.

Step Functions provide a clear monitoring of each execution, which is an advantage compared to non-Serverless solution. In a non-Serverless application we would need to add an additional functionality to reach the same result.

6. Conclusion

In this thesis we have introduced five different architectural design patterns, which are web applications, event-driven data processing, data lakes, and stream processing. It is important to mention, that each pattern can be implemented differently and composed of different services. These patterns are closely related to the benefits and drawbacks of Serverless computing and the decision of going Serverless should be based on considering them. The benefits are usually related to reducing costs and we have demonstrated two fields, which have an impact on costs. The first is human labour and the second is resource. Other advantages bring reduced risk and a flexibility of scaling. Serverless is not perfect and it can have an impact on the performance, which is affected by the latency and cold starts. With Serverless we also lose the control over various aspects i.e. configuration, performance, resolution, and security, which may be a problem in particular cases. A significant problem can be local testing, which is related to debugging. This is definitely an aspect, that has to be improved by the cloud providers.

The practical part of this thesis was concerned with a prototype application. We have described its functionality, technology environment, implementation, and we discussed the monitoring of the application. Also we have evaluated various aspects related to the application.

In most cases it is cheaper to execute AWS Lambda, than running a server constantly, but there is not enough studies, which confirms, that running a complex Serverless application, which includes multiple services, is still cost effective. The services, which are used for building a Serverless application are not a free service and therefore, the total costs could be surprising. Also, the cold starts and the latency have impact on the performance and to achieve a good performance is necessary to correctly design the applications. An interesting idea would be to compare the costs and performance of two Serverless applications, which would have the same functionality, but a different design. For instance, the prototype application, which was created for this thesis, works, but it is not sure, that this design is the cheapest solution with the best performance. The Step Functions could be replaced by API gateway, or the Lambda functions could trigger each other and it could be found out, which solution would be better.

References

1. ROBERTS, Michael; CHAPIN, John. *What is Serverless? Understanding the Latest Advances in Cloud and Service-Based Architecture*. Gravenstein Highway North, Sebastopol: O'Reilly Media, 2017. ISBN 978-1-491-98416-1. Available also from: <https://dl.illwieckz.net/u/thomas-debesse/bazar/oreilly/free/www.oreilly.com/programming/free/files/what-is-serverless.pdf>.
2. ZAMBRANO, Brian. *Serverless Design Patterns and Best Practices*. Birmingham, UK: Packt Publishing Limited, 2018. ISBN 9781788620642.
3. GUPTA, Mohit. *Serverless Architectures with AWS*. Birmingham, UK: Packt Publishing Limited, 2018. ISBN 9781789805024.
4. ROBERTS, Michael. *Serverless Architectures*. 2018. Available also from: <https://martinfowler.com/articles/serverless.html#WhatIsServerless>.
5. FRUHLINGER, Josh. What is serverless? Serverless computing explained. *Info World.com*. July 2019. Available also from: <https://www.infoworld.com/article/3406501/what-is-serverless-serverless-computing-explained.html>.
6. MELL, Peter; GRANCE, Tim. The NIST Definition of Cloud Computing. 2011. Available also from: <https://csrc.nist.gov/publications/detail/sp/800-145/final>.
7. SCHOUTEN, Edwin. Cloud computing defined: Characteristics service levels. 2014. Available also from: <https://www.ibm.com/blogs/cloud-computing/2014/01/31/cloud-computing-defined-characteristics-service-levels/>.
8. SOMEPALLE, Sankar. 3 Service and 4 Deployment Models of Cloud Computing. 2015. Available also from: <https://www.linkedin.com/pulse/3-service-4-deployment-models-cloud-computing-sankar-somepalle>.
9. DREW, Dennis; RANGANATH, Maitreya. *AWS re:Invent 2017: Serverless Architectural Patterns and Best Practices (ARC401)*. 2017. Available also from: https://www.youtube.com/watch?v=Xi_WrinvTnM&t.
10. PILLAI, Unni; MULLER, Daniel. *AWS Summit Singapore - Architecting a Serverless Data Lake on AWS*. 2018. Available also from: <https://www.youtube.com/watch?v=B1LuBN3fw3I&t=97s>.
11. DALY, Jeremy. *Serverless Microservice Patterns for AWS*. 2019. Available also from: <https://www.jeremydaly.com/serverless-microservice-patterns-for-aws/>.
12. HOANG, Andrew. Analysis of Micro Services and Serverless Architecture for Mobile Application Enablement. 2017. Available also from: <http://scholarworks.csun.edu/bitstream/handle/10211.3/193100/Hoang-Andrew-thesis-2017.pdf?sequence=1>.
13. DRAGONI, Nicola; GIALLORENZO, Saverio; LAFUENTE, Alberto Lluch; MAZZARA, Manuel; MONTESI, Fabrizio; MUSTAFIN, Ruslan; SAFINA, Larisa. Microservices: yesterday, today, and tomorrow. 2017. Available also from: <https://arxiv.org/pdf/1606.04036.pdf>.

14. VILLAMIZAR, Mario. Cost comparison of running web applications in the cloud using monolithic, microservice, and AWS Lambda architectures. 2016. Available also from: https://www.researchgate.net/profile/Mario_Villamizar/publication/316532483_Cost_comparison_of_running_web_applications_in_the_cloud_using_monolithic_microservice_and_AWS_Lambda_architectures/links/5b3acfb4a6fdcc8506ea9c7d/Cost-comparison-of-running-web-applications-in-the-cloud-using-monolithic-microservice-and-AWS-Lambda-architectures.pdf.
15. GHOSH, Bishakh Chandra; ADDYA, Sourav Kanti; SOMY, Nishant Baranwal; NATH, Shubha Brata; CHAKRABORTY, Sandip; GHOSH, Soumya K. Caching Techniques to Improve Latency in Serverless Architectures. 2019. Available also from: <https://arxiv.org/pdf/1911.07351.pdf>.
16. YAN, Mengting. Building a Chatbot with Serverless Computing. 2018. Available also from: <https://dl.acm.org/doi/pdf/10.1145/3007203.3007217>.
17. AMAZON WEB SERVICES. AWS Step Functions Use Cases. 2020. Available also from: <https://aws.amazon.com/step-functions/use-cases/>.
18. AMAZON WEB SERVICES. Serverless Streaming Architectures and Best Practices. 2018. Available also from: https://d1.awsstatic.com/whitepapers/Serverless_Streaming_Architecture_Best_Practices.pdf.
19. AMAZON WEB SERVICES. Activities. 2020. Available also from: <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-activities.html>.
20. AMAZON WEB SERVICES. Class: AWS.Lambda. 2020. Available also from: <https://docs.aws.amazon.com/AWSJavaScriptSDK/latest/AWS/Lambda.html>.
21. AMAZON WEB SERVICES. *Introducing AWS Lambda*. 2014. Available also from: <https://aws.amazon.com/about-aws/whats-new/2014/11/13/introducing-aws-lambda/>.
22. AMAZON WEB SERVICES. Introducing Amazon API Gateway. 2015. Available also from: <https://aws.amazon.com/about-aws/whats-new/2015/07/introducing-amazon-api-gateway/>.
23. AMAZON WEB SERVICES. AWS Lambda. Available also from: <https://aws.amazon.com/lambda/>.
24. FROMM, Ken. Why The Future Of Software And Apps Is Serverless. 2012. Available also from: <https://readwrite.com/2012/10/15/why-the-future-of-software-and-apps-is-serverless/>.
25. STANLEY, Ant. Servers are dead.. 2015. Available also from: <https://precipitation.io/servers-are-dead-3c0fa8d77259>.
26. NAGRAI, Vyom; SANDHAUS, Jay. (ARC308) The Serverless Company: Using AWS Lambda. 2015. Available also from: <https://www.slideshare.net/AmazonWebServices/arc308-the-serverless-company-using-aws-lambda>.
27. ISO/IEC TS 23167:2020(en). 2018. Available also from: <https://inteco.isolutions.iso.org/obp/ui#iso:std:iso-iec:ts:23167:ed-1:v1:en>.

28. AMAZON WEB SERVICES. What is serverless? 2020. Available also from: <https://aws.amazon.com/serverless/>.
29. SCHMIDT, Eric; SULLIVAN, Danny. Search Engine Strategies Conference. 2006. Available also from: <https://www.google.com/press/podium/ses2006.html>.
30. WOLF, Oliver. Serverless Architecture in Short. Available also from: <https://specify.io/concepts/serverless-baas-faas>.
31. What Is Function as a Service (FaaS)? 2020. Available also from: <https://www.cloudflare.com/learning/serverless/glossary/function-as-a-service-faas/>.
32. BALDINI, Ioana et al. Serverless Computing: Current Trends and Open Problems. 2017.
33. ALLEN, Sarah et al. CNCF Serverless Whitepaper v1.0. 2018. Available also from: https://github.com/cncf/wg-serverless/blob/master/whitepapers/serverless-overview/cncf_serverless_whitepaper_v1.0.pdf.
34. TANASA, Wisen. The traits of serverless architecture. 2019. Available also from: <https://www.thoughtworks.com/insights/blog/traits-serverless-architecture>.
35. MAZUR, Ola. Monolithic architecture vs microservices. 2020. Available also from: <https://divante.com/blog/monolithic-architecture-vs-microservices/>.
36. FERNANDO, Manor. Evolving Architecture | Serverless Patterns and Use Cases. 2018. Available also from: <https://medium.com/@manojf/serverless-patterns-and-use-cases-b30865758301>.
37. ROSSI, Ben. Data lakes vs data streams: which is better? 2016. Available also from: <https://www.information-age.com/data-lakes-vs-data-streams-which-is-better-123461535/>.
38. JAIN, Aakash. Introduction to AWS Step Function. 2018. Available also from: <https://blog.knoldus.com/introduction-to-aws-step-function/>.
39. BRAZEAL, Forrest. Reasons Not To Serverless - Think FaaS Podcastg. 2018. Available also from: <https://www.trek10.com/blog/think-faas-podcast-reasons-not-to-serverless>.
40. EDI BASICS. What is EDI (Electronic Data Interchange)? Available also from: <https://www.edibasics.com/what-is-edi/>.
41. ANYWHERE, GO. EDI x12 vs. EDIFACT: What's the difference? 2019. Available also from: <https://www.goanywhere.com/blog/edi-x12-vs-edifact-whats-the-difference>.
42. BERRYWORKS SOFTWARE. Available also from: <http://www.berryworkssoftware.net>.
43. SAXONICA. Available also from: <http://www.saxonica.com/welcome/welcome.xml>.

Appendix

A. Source Code of the Prototype Application

A.1 Step Functions Workflow

The following code defines the workflow of the Step Functions.

```
1  {
2    "StartAt": "GetFileType",
3    "States": {
4      "GetFileType": {
5        "Type": "Task",
6        "Resource": "arn:aws:lambda:eu-central-1:630997192154:function:getFileType",
7        "TimeoutSeconds": 5,
8        "ResultPath": "\$.results.fileType",
9        "Next": "CheckFileType",
10       "Catch": [
11         {
12           "ErrorEquals": [
13             "States.ALL"
14           ],
15           "Next": "Failure"
16         }
17       ],
18     },
19     "CheckFileType": {
20       "Type": "Choice",
21       "Choices": [
22         {
23           "Variable": "\$.results.fileType",
24           "StringEquals": "xml",
25           "Next": "UploadXMLToS3"
26         },
27         {
28           "Variable": "\$.results.fileType",
29           "StringEquals": "edi",
30           "Next": "UploadEDIToS3"
31         },
32         {
33           "Variable": "\$.results.fileType",
34           "StringEquals": "xsl",
35           "Next": "UploadXSLTtoS3"
36         }
37       ],
38       "Default": "Failure"
39     },
40     "Failure": {
41       "Type": "Fail"
```

```

42     },
43     "UploadXSLTtoS3": {
44         "Type": "Task",
45         "Resource": "arn:aws:lambda:eu-central-1:630997192154:function:UploadXSLtoS3",
46         "Next": "DeleteSourceFile"
47     },
48     "UploadXMLToS3": {
49         "Type": "Task",
50         "Resource": "arn:aws:lambda:eu-central-1:630997192154:function:UploadXMLtoS3",
51         "Next": "GetXSLTmap"
52     },
53     "UploadEDITtoS3": {
54         "Type": "Task",
55         "Resource": "arn:aws:lambda:eu-central-1:630997192154:function:UploadEDITtoS3",
56         "Next": "EDITtoXML"
57     },
58     "EDITtoXML": {
59         "Type": "Task",
60         "Resource": "arn:aws:states:eu-central-1:630997192154:activity:EDITtoXML",
61         "Next": "GetXSLTmap"
62     },
63     "XSLTTransformation": {
64         "Type": "Task",
65         "Resource": "arn:aws:lambda:eu-central-1:630997192154:function:TransformXML",
66         "Next": "DeleteSourceFile"
67     },
68     "GetXSLTmap": {
69         "Type": "Pass",
70         "Next": "XSLTTransformation"
71     },
72     "DeleteSourceFile": {
73         "Type": "Task",
74         "Resource": "arn:aws:lambda:eu-central-1:630997192154:function:deleteSourceFile",
75         "End": true
76     }
77 }
78 }

```

A.2 Lambda Functions

A.2.1 Invoke Step Functions

The following code is triggered by S3 event PUT and invokes the Step Functions workflow.

```

1  const AWS = require('aws-sdk');
2  AWS.config.update({region: 'eu-central-1'});
3  const stepFunction = new AWS.StepFunctions();
4
5  exports.handler = async (event) => {

```

```

6     let uploadedFiles = event.Records.map(async(record) => {
7         let params = {
8             stateMachineArn: process.env.STATE_MACHINE_ARN,
9             input: JSON.stringify(record)
10        };
11        let data = await stepFunction.startExecution(params).promise();
12        return data;
13    });
14
15    let results = await Promise.all(uploadedFiles);
16    return results;
17 };

```

A.2.2 Get File Type

The following code get the type of the file from the file name.

```

1  exports.handler = async (event) => {
2      let filename = event.s3.object.key;
3      let index = filename.lastIndexOf('.');
4
5      if (index > 0)
6          return filename.substring(index + 1);
7
8      else {
9          return null;
10     }
11 };

```

A.2.3 Upload XSLT to S3

The following code copies the XSLT file from the source bucket to the destination bucket.

```

1  const AWS = require('aws-sdk');
2  AWS.config.update({region: 'eu-central-1'});
3
4  const s3 = new AWS.S3();
5
6  exports.handler = async (event) => {
7      let params = {
8          Bucket: process.env.DESTINATION_BUCKET,
9          CopySource: encodeURIComponent('/') + event.s3.bucket.name + '/' + event.s3.object.key),
10         Key: event.s3.object.key
11     };
12
13     await s3.copyObject(params).promise();
14
15     return {

```

```

16         region: 'eu-central-1',
17         bucket: process.env.DESTINATION_BUCKET,
18         Key: event.s3.object.key
19     }
20 };

```

A.2.4 Upload XML to S3

The following code copies the XML file from the source bucket to the destination bucket.

```

1  const AWS = require('aws-sdk');
2  AWS.config.update({region: 'eu-central-1'});
3
4  const s3 = new AWS.S3();
5
6  exports.handler = async (event) => {
7      let params = {
8          Bucket: process.env.DESTINATION_BUCKET,
9          CopySource: encodeURI('/'+ event.s3.bucket.name + '/' + event.s3.object.key),
10         Key: event.s3.object.key
11     };
12
13     await s3.copyObject(params).promise();
14
15     return {
16         region: 'eu-central-1',
17         bucket: process.env.DESTINATION_BUCKET,
18         key: event.s3.object.key
19     }
20 };

```

A.3 Upload EDI to S3

The following code copies the EDI file from the source bucket to the destination bucket.

```

1  const AWS = require('aws-sdk');
2  AWS.config.update({region: 'eu-central-1'});
3
4  const s3 = new AWS.S3();
5
6  exports.handler = async (event) => {
7      let params = {
8          Bucket: process.env.DESTINATION_BUCKET,
9          CopySource: encodeURI('/'+ event.s3.bucket.name + '/' + event.s3.object.key),
10         Key: event.s3.object.key
11     };
12

```

```

13     await s3.copyObject(params).promise();
14
15     return {
16         region: 'eu-central-1',
17         bucket: process.env.DESTINATION_BUCKET,
18         key: event.s3.object.key
19     }
20 };

```

A.3.1 Delete File

This code deletes any kind of file from the source bucket.

```

1  const AWS = require('aws-sdk');
2  AWS.config.update({region: 'eu-central-1'});
3  const s3 = new AWS.S3();
4
5  exports.handler = async (event) => {
6      let params = {
7          Bucket: process.env.ENV_BUCKET,
8          Key: event.Key
9      };
10
11      await s3.deleteObject(params).promise();
12
13      return true;
14  };

```

A.3.2 XSLT Transformation

This code transforms the XML file into another XML file using the XSLT stylesheet. It has multiple classes.

Class Handler

Class handler is the first class, which is invoked and the function *handleRequest()* takes the input, process it, and send an output.

```

1  import com.amazonaws.services.lambda.runtime.Context;
2  import com.amazonaws.services.lambda.runtime.RequestStreamHandler;
3  import org.json.simple.JSONObject;
4  import org.json.simple.parser.JSONParser;
5  import s3.S3Processor;
6  import xslt.XSLTProcessor;
7
8  import javax.xml.transform.TransformerException;
9  import java.io.*;

```



```

10
11 public class Handler implements RequestStreamHandler {
12     @Override
13     public void handleRequest(InputStream inputStream, OutputStream outputStream, Context context) throws IOException {
14
15         JSONObject event = new JSONObject();
16         JSONParser parser = new JSONParser();
17         StringBuilder builder = new StringBuilder();
18         BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(inputStream));
19         try {
20             event = (JSONObject) parser.parse(bufferedReader);
21         } catch (org.json.simple.parser.ParseException e) {
22             e.printStackTrace();
23         }
24
25         String xml = "";
26         String result = "";
27
28         xml = event.get("key").toString();
29         S3Processor s3 = new S3Processor();
30         XSLTProcessor xsltProcessor = new XSLTProcessor();
31         String content = null;
32
33         try {
34             content = xsltProcessor.transform(s3.getXslFile("dw.xsl"), s3.getXmlFile(xml));
35             s3.upload(content, "new_" + xml);
36             result = "Successful";
37             builder.append("{");
38             builder.append("\"Result\": \"" + result + "\"");
39             builder.append(",");
40             builder.append("\"Key\": \"" + xml + "\"");
41             builder.append("}");
42             OutputStreamWriter writer = new OutputStreamWriter(outputStream, "UTF-8");
43             writer.write(builder.toString());
44             writer.close();
45         } catch (TransformerException e) {
46             e.printStackTrace();
47         }
48     }
49 }

```

Class XSLT Processor

This class contains the essential code for the transformation as such. It is prepared for processing additional parameters and imports. However, for our purpose we have not enabled this functionality.

```

1 import javax.xml.transform.Transformer;
2 import javax.xml.transform.TransformerException;
3 import javax.xml.transform.TransformerFactory;
4 import javax.xml.transform.stream.StreamResult;

```

```

5  import javax.xml.transform.stream.StreamSource;
6  import java.io.ByteArrayInputStream;
7  import java.io.StringWriter;
8  import java.util.HashMap;
9  import java.util.Set;
10
11 public class XSLTProcessor {
12
13     private HashMap<String, Integer> parameters;
14     private Config config = null;
15
16
17     public void setParameters(HashMap<String, Integer> parameters) {
18         this.parameters = parameters;
19     }
20
21     public void setUriResolver(HashMap<String, byte[]> imports) {
22         config = new Config(imports);
23     }
24
25     public String transform(byte[] xslt, byte[] xml) throws TransformerException {
26
27         TransformerFactory tFactory;
28
29         if (config == null) {
30             tFactory = TransformerFactory.newInstance("net.sf.saxon.TransformerFactoryImpl", null);
31         } else {
32             tFactory = config.getFactory();
33         }
34
35         final StringWriter writer = new StringWriter();
36         Transformer transformer = tFactory.newTransformer(new StreamSource(new ByteArrayInputStream(xslt)));
37
38         if (parameters != null) {
39             Set<String> names = parameters.keySet();
40             for (String name : names) {
41                 transformer.setParameter(name, parameters.get(name));
42             }
43         }
44
45         StreamResult standardResult = new StreamResult(writer);
46         transformer.transform(new StreamSource(new ByteArrayInputStream(xml)), standardResult);
47         return writer.toString();
48     }
49 }

```

Class Configuration

This class processes the parameters and imports. This functionality is not enabled in the workflow.

```

1  import javax.xml.transform.Source;

```

```

2  import javax.xml.transform.TransformerException;
3  import javax.xml.transform.TransformerFactory;
4  import javax.xml.transform.URIResolver;
5  import javax.xml.transform.stream.StreamSource;
6  import java.io.ByteArrayInputStream;
7  import java.util.HashMap;
8  import java.util.Set;
9
10 public class Config {
11
12
13     private TransformerFactory tFactory;
14     private HashMap<String, byte[]> imports;
15
16     protected Config(HashMap<String, byte[]> imports) {
17         tFactory = TransformerFactory.newInstance("net.sf.saxon.TransformerFactoryImpl", null);
18         this.imports = imports;
19         tFactory.setURIResolver(resolver);
20     }
21
22     private URIResolver resolver = new URIResolver() {
23         @Override
24         public Source resolve(String href, String base) throws TransformerException {
25             Set<String> keySet = imports.keySet();
26             for (String key : keySet) {
27                 if (href.contains(key)) {
28                     byte[] stylesheet = imports.get(key);
29                     return new StreamSource(new ByteArrayInputStream(stylesheet));
30                 }
31             }
32             return null;
33         }
34     };
35
36
37     protected TransformerFactory getFactory() {
38         return tFactory;
39     }
40
41 }

```

Class S3 Processor

This class enables handling files on S3 buckets.

```

1  import com.amazonaws.services.s3.AmazonS3;
2  import com.amazonaws.services.s3.AmazonS3ClientBuilder;
3  import com.amazonaws.services.s3.model.DeleteObjectRequest;
4  import com.amazonaws.services.s3.model.GetObjectRequest;
5  import com.amazonaws.services.s3.model.S3Object;
6
7  import java.io.InputStream;

```

```

8  import java.util.Scanner;
9
10 public class S3Processor {
11
12     private AmazonS3 s3Client;
13     private static String SOURCEBUCKETXML;
14     private static String SOURCEBUCKETXMLFIRST;
15     private static String SOURCEBUCKETXSLT;
16     private static String DESTINATIONBUCKET;
17
18
19     public S3Processor() {
20         s3Client = AmazonS3ClientBuilder
21             .standard()
22             .withRegion("eu-central-1")
23             .build();
24         SOURCEBUCKETXML = "xmlfilesformybt";
25         SOURCEBUCKETXMLFIRST = "jsonfilesformybt";
26         SOURCEBUCKETXSLT = "xsltmaps";
27         DESTINATIONBUCKET = "finalxmlfiles";
28     }
29
30     public boolean upload(String content, String xmlName) {
31         s3Client.putObject(DESTINATIONBUCKET, xmlName, content);
32         return false;
33     }
34
35     public byte[] getXmlFile(String fileName) {
36
37         S3Object s3Object = s3Client.getObject(new GetObjectRequest(SOURCEBUCKETXML, fileName));
38         InputStream objectData = s3Object.getObjectContent();
39         Scanner scanner = new Scanner(objectData);
40         String file = "";
41         while (scanner.hasNext()) {
42             file += scanner.nextLine();
43         }
44         scanner.close();
45
46         if (file != null) {
47             return file.getBytes();
48         } else {
49             return null;
50         }
51     }
52
53     public byte[] getXslFile(String fileName) {
54
55         S3Object s3Object = s3Client.getObject(new GetObjectRequest(SOURCEBUCKETXSLT, fileName));
56         InputStream objectData = s3Object.getObjectContent();
57         Scanner scanner = new Scanner(objectData);
58         String file = "";
59         while (scanner.hasNext()) {
60             file += scanner.nextLine();

```

```

61     }
62     scanner.close();
63
64     if (file != null) {
65         return file.getBytes();
66     } else {
67         return null;
68     }
69 }
70 }

```

Pom.xml

This file contains dependencies and the configuration for building the JAR file.

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
6
7      <groupId>droz00</groupId>
8      <artifactId>XSLTActivityProcessor</artifactId>
9      <version>1.0-SNAPSHOT</version>
10     <build>
11         <plugins>
12             <plugin>
13                 <groupId>org.apache.maven.plugins</groupId>
14                 <artifactId>maven-jar-plugin</artifactId>
15                 <configuration>
16                     <archive>
17                         <manifest>
18                             <mainClass>Main</mainClass>
19                         </manifest>
20                     </archive>
21                 </configuration>
22             </plugin>
23             <plugin>
24                 <groupId>org.apache.maven.plugins</groupId>
25                 <artifactId>maven-compiler-plugin</artifactId>
26                 <configuration>
27                     <source>1.8</source>
28                     <target>1.8</target>
29                 </configuration>
30             </plugin>
31             <plugin>
32                 <artifactId>maven-shade-plugin</artifactId>
33                 <version>3.2.1</version>
34                 <configuration>
35                     <filters>
36                         <filter>
37                             <artifact>*:*</artifact>

```

```

38         <excludes>
39             <exclude>META-INF/*.SF</exclude>
40             <exclude>META-INF/*.DSA</exclude>
41             <exclude>META-INF/*.RSA</exclude>
42         </excludes>
43     </filter>
44 </filters>
45 </configuration>
46 <executions>
47     <execution>
48         <phase>package</phase>
49         <goals>
50             <goal>shade</goal>
51         </goals>
52     </execution>
53 </executions>
54 </plugin>
55 </plugins>
56 </build>
57
58 <dependencies>
59     <dependency>
60         <groupId>net.sf.saxon</groupId>
61         <artifactId>Saxon-HE</artifactId>
62         <version>9.9.1-2</version>
63     </dependency>
64     <dependency>
65         <groupId>com.amazonaws</groupId>
66         <artifactId>aws-java-sdk-s3</artifactId>
67         <version>1.11.584</version>
68     </dependency>
69     <dependency>
70         <groupId>com.amazonaws</groupId>
71         <artifactId>aws-lambda-java-core</artifactId>
72         <version>1.2.0</version>
73     </dependency>
74     <dependency>
75         <groupId>com.amazonaws</groupId>
76         <artifactId>aws-lambda-java-events</artifactId>
77         <version>2.2.6</version>
78     </dependency>
79     <dependency>
80         <groupId>com.googlecode.json-simple</groupId>
81         <artifactId>json-simple</artifactId>
82         <version>1.1</version>
83     </dependency>
84 </dependencies>
85 </project>

```

A.4 Activity Worker

A.4.1 EDI Transformation

This is the code of the Activity Worker, which is running on an EC2 instance.

Class Main

This class is the first class of this code, which is executed.

```
1 import activity.ActivityProcessor;
2
3 import java.io.IOException;
4
5 public class Main {
6     public static void main(String[] args) throws IOException {
7         new ActivityProcessor().listen();
8     }
9 }
```

Class Activity Processor

This class waits for the requests sent from the Step Function. It processes the file and returns either success response, or failure response.

```
1 import com.amazonaws.ClientConfiguration;
2 import com.amazonaws.auth.AWSSStaticCredentialsProvider;
3 import com.amazonaws.auth.BasicAWSCredentials;
4 import com.amazonaws.services.stepfunctions.AWSSStepFunctions;
5 import com.amazonaws.services.stepfunctions.AWSSStepFunctionsClientBuilder;
6 import com.amazonaws.services.stepfunctions.model.GetActivityTaskRequest;
7 import com.amazonaws.services.stepfunctions.model.GetActivityTaskResult;
8 import com.amazonaws.services.stepfunctions.model.SendTaskFailureRequest;
9 import com.amazonaws.services.stepfunctions.model.SendTaskSuccessRequest;
10 import com.amazonaws.util.json.Jackson;
11 import com.fasterxml.jackson.databind.JsonNode;
12 import editoxml.EDItoXMLprocessor;
13 import s3.S3processor;
14
15 import java.util.concurrent.TimeUnit;
16
17 public class ActivityProcessor {
18
19     private static String ARN = "arn:aws:states:eu-central-1:630997192154:activity:EDItoXML";
20     private static String ACCESS_KEY = "AKIAZF2S5BXNJ5JOS2UO";
21     private static String SECRET_KEY = "jdBraD+EQ/Dw+n905pn";
22     BasicAWSCredentials credentials = new BasicAWSCredentials(ACCESS_KEY, SECRET_KEY);
23
24     public void listen() {
```

```

25     final ClientConfiguration clientConfiguration = new ClientConfiguration();
26     clientConfiguration.setSocketTimeout((int) TimeUnit.SECONDS.toMillis(10000));
27
28     final AWSStepFunctions client = AWSStepFunctionsClientBuilder
29         .standard()
30         .withRegion("eu-central-1")
31         .withCredentials(new AWSStaticCredentialsProvider(credentials))
32         .build();
33
34     String edi;
35     String result;
36     String fileName;
37
38     System.out.println("Listening...");
39
40     while (true) {
41         GetActivityTaskResult getActivityTaskResult = client.getActivityTask(
42             new GetActivityTaskRequest().withActivityArn(ARN));
43
44         if (getActivityTaskResult != null) {
45
46
47             try {
48                 final JsonNode json = Jackson.jsonNodeOf(getActivityTaskResult.getInput());
49                 if (json != null) {
50                     if (json.get("key") != null) {
51                         System.out.println("Processing event...");
52                         fileName = json.get("key").textValue();
53                         S3processor s3 = new S3processor();
54                         edi = s3.get(fileName);
55
56                         s3.upload(new EDItoXMLprocessor().transform(edi), fileName);
57
58                         result = process(fileName);
59                         client.sendTaskSuccess(new SendTaskSuccessRequest()
60                             .withOutput(result)
61                             .withTaskToken(getActivityTaskResult.getTaskToken()));
62                     }
63                 }
64             } catch (Exception e) {
65                 e.printStackTrace();
66
67                 client.sendTaskFailure(new SendTaskFailureRequest()
68                     .withTaskToken(getActivityTaskResult.getTaskToken()));
69             }
70         } else {
71             try {
72                 Thread.sleep(1000);
73             } catch (InterruptedException e) {
74                 e.printStackTrace();
75             }
76         }
77     }

```



```

78     }
79
80     private String process(final String file) {
81         System.out.println("Successful");
82         return "{\"key\": \"" + file + "\"}";
83     }
84 }
85 }

```

Class S3 Processor

This class handles the files on S3 bucket.

```

1  package s3;
2
3  import com.amazonaws.services.s3.AmazonS3;
4  import com.amazonaws.services.s3.AmazonS3ClientBuilder;
5  import com.amazonaws.services.s3.model.DeleteObjectRequest;
6  import com.amazonaws.services.s3.model.GetObjectRequest;
7  import com.amazonaws.services.s3.model.S3Object;
8
9  import java.io.InputStream;
10 import java.util.Scanner;
11
12 public class S3processor {
13
14     private AmazonS3 s3Client;
15     private static String SOURCEBUCKETFIRST;
16     private static String SOURCEBUCKET;
17     private static String DESTINATIONBUCKET;
18     private String file;
19
20     public S3processor() {
21         s3Client = AmazonS3ClientBuilder
22             .standard()
23             .withRegion("eu-central-1")
24             .build();
25         SOURCEBUCKETFIRST = "jsonfilesformybt";
26         SOURCEBUCKET = "edimessage";
27         DESTINATIONBUCKET = "xmlfilesformybt";
28         file = "";
29     }
30
31
32
33     public boolean upload(String content, String fileName) {
34         s3Client.putObject(DESTINATIONBUCKET, fileName, content);
35         return false;
36     }
37
38     public String get(String fileName) {
39

```

```

40         S3Object s3Object = s3Client.getObject(new GetObjectRequest(SOURCEBUCKET, fileName));
41         InputStream objectData = s3Object.getObjectContent();
42         Scanner scanner = new Scanner(objectData);
43         while (scanner.hasNext()) {
44             file += scanner.nextLine();
45         }
46         scanner.close();
47
48         if (file != null) {
49             return file;
50         } else {
51             return null;
52         }
53     }
54
55     public boolean delete(String fileName) {
56         s3Client.deleteObject(new DeleteObjectRequest(SOURCEBUCKET, fileName));
57         s3Client.deleteObject(new DeleteObjectRequest(SOURCEBUCKETFIRST, fileName));
58         return true;
59     }
60 }

```

Class EDI Processor

This is the essential code, which transforms the EDI file files into XML.

```

1  package editoxml;
2
3  import com.berryworks.edireader.demo.EDItToXML;
4
5  import java.io.StringReader;
6  import java.io.StringWriter;
7
8  public class EDItToXMLprocessor {
9
10
11     private StringReader reader;
12     private StringWriter writer;
13
14     public String transform(String edi){
15         reader = new StringReader(edi);
16         writer = new StringWriter();
17         new EDItToXML(reader,writer).run();
18         return writer.toString();
19     }
20 }

```

Pom.xml

This file contains dependencies and the configuration for building the JAR file.

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
6
7      <groupId>droz00</groupId>
8      <artifactId>EDItToXML2</artifactId>
9      <version>1.0-SNAPSHOT</version>
10
11     <build>
12         <plugins>
13             <plugin>
14                 <groupId>org.apache.maven.plugins</groupId>
15                 <artifactId>maven-jar-plugin</artifactId>
16                 <configuration>
17                     <archive>
18                         <manifest>
19                             <mainClass>Main</mainClass>
20                         </manifest>
21                     </archive>
22                 </configuration>
23             </plugin>
24             <plugin>
25                 <groupId>org.apache.maven.plugins</groupId>
26                 <artifactId>maven-compiler-plugin</artifactId>
27                 <configuration>
28                     <source>1.8</source>
29                     <target>1.8</target>
30                 </configuration>
31             </plugin>
32             <plugin>
33                 <artifactId>maven-shade-plugin</artifactId>
34                 <version>3.2.1</version>
35                 <configuration>
36                     <filters>
37                         <filter>
38                             <artifact>*:*</artifact>
39                             <excludes>
40                                 <exclude>META-INF/*.SF</exclude>
41                                 <exclude>META-INF/*.DSA</exclude>
42                                 <exclude>META-INF/*.RSA</exclude>
43                             </excludes>
44                         </filter>
45                     </filters>
46                 </configuration>
47                 <executions>
48                     <execution>
49                         <phase>package</phase>
50                         <goals>
51                             <goal>shade</goal>
52                         </goals>
53                     </execution>

```

```

54         </executions>
55     </plugin>
56 </plugins>
57 </build>
58
59 <dependencies>
60     <!-- https://mvnrepository.com/artifact/com.amazonaws/aws-java-sdk-stepfunctions -->
61     <dependency>
62         <groupId>com.amazonaws</groupId>
63         <artifactId>aws-java-sdk-stepfunctions</artifactId>
64         <version>1.11.682</version>
65     </dependency>
66     <!-- https://mvnrepository.com/artifact/com.amazonaws/aws-java-sdk-s3 -->
67     <dependency>
68         <groupId>com.amazonaws</groupId>
69         <artifactId>aws-java-sdk-s3</artifactId>
70         <version>1.11.584</version>
71     </dependency>
72     <dependency>
73         <groupId>com.berryworks</groupId>
74         <artifactId>edireader</artifactId>
75         <version>4.7.3.1</version>
76     </dependency>
77 </dependencies>
78
79 </project>

```